

ARCHON AGENTIC OS 2026 . FEATURELISTE COMPLETATO

Catalogo di sistema e modulo, panoramica dell'architettura e matrice funzionale

Versione: QX0ZQ (- .Archon Hardened- - .) . . . . . .

Catalogo di sistema e modulo, panoramica dell'architettura e matrice funzionale

© Michael Keller

Sommario

1. Versionsstand & Release-Übersicht (v0.5.1) 3
2. System-Ansichten & Spezial-Studios (UI Views) 3
3. Token-Spar- & FinOps-Ökosystem (Kostenminimierung) 8
4. Kernel, Scheduling & Daemon-Architektur (Pillar 1) 11
5. Multi-Agenten-Prozessmodell, Hierarchie & Synthese (Pillar 2) 12
6. MCP-Treiber, Externe Dienste & Multimodale Werkzeuge (Pillar 3) 14
7. Gedächtnis, Datenbank-Hygiene & Obsidian 2nd Brain (Pillar 4) 15
8. Sicherheit, Zero-Trust Governance & System-Diagnose (Pillar 6) 17
9. Selbst-Evolution, Marktplätze & Auto-Dokumentation (Pillar 7) 18
10. Referenz-Matrix: Steuerbare Evey/Hermes-Plugins 19
11. Enterprise- & Resilienz-Architektur 21
12. Autonomie-, Sandbox- & Governance-Architektur 23
13. Performance-Core, Durchsatz-Skalierung & Token-Effizienz 24
14. Moderne Gedächtnis- & RAG-Architektur 25
15. Zuverlässige Ausführung & Determinismus (Durable Execution) 27
16. Sicherheit, Datenschutz & Enterprise-Guardrails 28
17. Selbst-Optimierung, Evaluation & Schwarm-Intelligenz 29
18. Säule 3: Wirtschaft & ERP (alpha) & Mandanten-Suite 31
19. Architektur-, Security- & Deployment-Features (v0.5.0 – v0.5.1) 31
20. Alphabetisches Glossar & Stichwortverzeichnis 32

1. Panoramica versione e rilascio (QX0ZQ)

Versione	Nome in codice	Migliaia di architettura e caratteristiche essenziali
QX0ZQ	Arcon Hardened	Router FastAPI modulare (main.py -88%), 25 risultati critici + 68 elevati di sicurezza corretti, Docker Admin di classe A, RBAC di gestione degli utenti (PBKDF2), 2 fasi aggiornamento di sistema (? local_only =true), QX0ZQ di integrazione proxy, installatore portatile 1-click.
QX0ZQ	Economia & QX0ZQ (alfa)	Colonna 3 completa (QX2ZQ / QX0ZQ - Sync, articoli/servizi, operazioni ANG /AB, contabilità con QX3ZQ & GiroCode, società madri), QX1ZQ v2 & 55 Frameworks.
QX0ZQ .x	Sentinel / Foundation	7 colonne kernel OS, 70 squadre di specialisti, 34+ plugin Evey, finOps 3 animali, sandbox ephemeral codice, replay debugging, PII -Scrobber, Semantic Cache & FTS5 Hybrid Search.

2. Visualizza sistemi e studi speciali (QX0ZQ Views)

1. Vista: QX0ZQ (Monitoring Cockpit & Leader)

Problema: Senza cockpit in tempo reale centrale, mancano la panoramica di agenti attivi, flussi di token, costi giornalieri e autorizzazioni bloccate.

Meccanica: Include 5 piastrelle KPI vitali (carico di flotta, portata TPM , guardia dei costi, stabilità, accesso di approval), la proiezione a 5 colonne, il Active Process Inspection Bar (3-Slot Footer per Heartbeat, Tool Concurrency e Context Saturation), la banda di attività di 15 minuti di token (Sparkline SVG) e il flusso di audit dal vivo.

Benefici: *Fornisce un monitoraggio a 360° senza ritardi dell'intero sistema operativo su un unico schermo.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

2. Vista: Operations . Kanban Board & Task Engine

Problema: Le attività in chat sono svolte in modo puramente informale e non possono essere controllate in base allo status o alle responsabilità.

Meccanica: 5 colonne del ciclo di vita (To-Do, In Edition, Review & QA, Bloccato, Chiuso) direttamente su kanban.db (Single Source of Truth). Supporta limiti di WIP configurabili per colonna, filtro di priorità (P0.. P3), assegnazione di agenti, cronologia e collegamento QX0ZQ a 1 clic. Integra il gate di qualità a 4 occhi: prima di spostare un compito su 'done', un revisionetore tra pari (Katharina Völk per QM/Compatibilità o Robert) controlla il risultato del lavoro per completezza, sostanza e capacità di azione (1...100% Punteggio). In 'changes_requested ', un intervallo di stato, un override umano in-the-Loop (force=true) sovraccarica se necessario.

Benefici: *Struttura tutti i progetti operativi in modo trasparente e previene la congestione del sistema attraverso le barriere Work-in Progress.* Impegna i posti di lavoro incompiuti nello stato 'done' attraverso una revisione obbligatoria a quattro occhi.

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

3. Vista: Operazioni . Organigramma e linee di report interattivi 2D

Problema: Le liste piatte di agenti non mostrano alcun mandato o sentieri di escalation in processi bloccati.

Meccanica: Visualizza 56 nodi e 55 linee di report come grafico 2D interattivo (Apex -> CEO Robert -> Leads specializzati -> Specialisti). Supporta trascinamento & drop per impiccare superiori, allineamento raster, visualizzazione scheda di squadra e memorizzazione permanente in config/ org_hierarchy .json.

Benefici: *Definisce catene di escalation e di delegazione obbligatorie che il sistema segue automaticamente in*

caso di blocco.

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

4. Vista: Operazioni . Comunicazione degli agenti interni (comunicazione radio)

Problema: Quando gli specialisti affidano tra loro compiti parziali, questo inonda la chat dell'utente o rimane invisibile.

Meccanica: Centro di intercettazione dedicato con lista canali (#generale, #tech-dev, #legal, #marketing) e feed di notizie cronologiche di tutti gli inter-agente di kanban.db.

Benefici: *Rende comprensibile la collaborazione della flotta degli agenti senza appesantire la cassetta postale primaria.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

5. Visualizza: Ingresso e Triage (3- colonne HITL - Gatekeeper & Postfach)

Problema: Le operazioni di scrittura non controllate di agenti (ad esempio la spedizione di posta elettronica) e gli ingressi di posta e telegramma dispersi comportano rischi per la sicurezza.

Meccanica: Triage centrale a tre colonne: colonna 1 per le autorizzazioni umane in the loop segnate in giallo (rilasciate/rifiutate), colonna 2 per la sincronizzazione di posta elettronica IMAP / SMTP con auto-triage e delegazione, colonna 3 per il flusso di telegram live specchiato (@Beatman_bot).

Benefici: *Assicura che non venga eseguita alcuna operazione di scrittura esterna senza l'esplicito rilascio dell'operatore.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

6. Vista: Agente & Gestione flotta (8-cassetta cockpit)
Problema: Gli specialisti hanno bisogno di modelli individuali, prompt di sistema, strumenti e istruzioni di lavoro in un unico luogo centrale.
Meccanica: Gestisce 70 profili specialistici con filtri rapidi (attivi, preferiti, tutti, 7 domini), swap hot per provider & modello e 8 scheda riprofessionale: panoramica (dossier 360° e nome di chiamata Telegram), chat diretta, abilità (skill), operazioni (story task), SOUL (editor di sistema prompt), file personali (SOP), mercato del talento e mercato degli skill.
Vantaggi: Dipartimento Personale centrale per la taratura e l'espansione seconde dell'intero ceppo agente.
Prezzo & lato ombra (trade-off): come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (richiesta di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se vengono eseguite azioni LLM attive all'interno del modulo).

7. Vista: Master Chat (doppia modalità Workspace & multi-sessione-sidebar)

Problema: Quando si lavora direttamente con gli agenti mancava un ambiente a schermo intero focalizzato con un rapido passaggio tra 1:1 dialogo, chat di gruppo e missione.

Meccanica: Fornisce lo switch dual-mode ('Diretto' per la risposta immediata contro 'Missione' per l'impianto di task strutturato), upload multi-file (fino a 5 file), cappa schermo del browser (Putton della fotocamera), inserimento vocale (diktat), calcolatore di bilancio reasoned (0-16k token), ottimizzazione di avvio dell'IA e storia di sessione destra (Agente vs. gruppo). Include il gasdotto Telegram Voice: messaggi vocali in arrivo (.ogg/opus) da quando si è in viaggio vengono trascritti via Whisper/Gemini, con nomi di chiamata (/robert, /avvocato, ecc.) inviati all'agente specializzato e risposte via edge-tts (de-DE-ConradNeural) come memo di lingua naturale direttamente sul telegramma.

Benefici: *Unisce il dialogo ad alta velocità, l'ingresso multimodale e il controllo della missione in un'area di*

lavoro continua. Consente il controllo vocale mobile completo e lo scambio audio bidirezionale tramite telegramma.

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

8. Visualizza: AI Mastermind & Boardroom (55 Frameworks metodologici)

Problema: decisioni strategiche complesse richiedono metodi di gestione strutturati e prospettive controverse piuttosto che opinioni individuali.

Meccanica: simula riunioni di forum multilivello (Chairman + 3 reali agenti specializzati + challenger Red Team) utilizzando 34 framework selezionabili in 7 categorie (35 base) (ad esempio CLEAR , Design Sprint, Double Diamond, Lean Startup, First Principles, Game Theory, Blue Ocean, Pre-Mortem, Six Sigma).

Benefici: Ammucchia rigore metodologico, rivela le macchie cieche dal team Red e fornisce memos decision-mature.

Prezzo & lato ombra (trade-off): come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (richiesta di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se vengono eseguite azioni LLM attive all'interno del modulo).

9. Vista: QX0ZQ (Explorer del progetto, galleria di artefatto e sandbox Python)

Problema: Rapporti generati, prototipi HTML, calcoli e script vengono persi in lunghe conversazioni.

Meccanica: Catalizza tutti i risultati del lavoro per tipo, data e produttore. Include Workspace Vault Reader con rendering dal vivo per Markdown/ HTML / QX0ZQ e una sandbox Python isolata per l'esecuzione sicura del codice. Avanzato intorno al runner Live QX0ZQ interattivo: consente l'esecuzione diretta di file Python e Code nel browser con timeout 12s, directory di scatch isolato e drawer terminale pieghevole (STDOUT , STDERR , codice Exit e durata di millisecondi).

Benefici: *Servito come sala di accettazione e presentazione per tutti i deliveryables finiti prima dell'uso del cliente o della produzione.* Gli artefatti e gli script di codice possono essere verificati e testati direttamente nel QX0ZQ senza cambiare console.

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

10. Visualizza: Biblioteca centrale & 2nd Brain (QX1ZQ , Auto- OCR & QX0ZQ - Vault)
Problema: Libri tecnici, norme DIN e piani di costruzione come PDF non sono stati ancora controllati dagli agenti senza copia manuale.
Meccanica: Unisce 3 sotto-tab e 4 core hub (00_HR_Agente, 02_Biblioteca della conoscenza, Protocolli/, 00_Notebook_Sintesi): 1. QX0ZQ RAG -indice (porta 3001), 2. Archiviatore documento con drag & drop, auto- OCR (docling) e classificazione tema IA, 3. albero file & markdown diretto QX1ZQ, incluso interruttore vault.
Vantaggi: Trasforma il PDF in conoscenza di markdown e vettori ricercabile per tutti i 70 agenti.
Prezzo & lato ombra (trade-off): come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (richiesta di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se vengono eseguite azioni LLM attive all'interno del modulo).

11. Visualizza: motore di flusso di lavoro (QX0ZQ Automation Mesh su porta 5678)

Problema: I processi di interfaccia ricorrenti (cron-job, webhook, bilanciamento di database) consumano token inutili quando sono in esecuzione su LLM.

Meccanica: Inserire direttamente l'editor n8n-Workflow visivo e collegarlo con l'agente CEO Robert tramite l'abilità n8n-automation-mesh (FastMCP). Rotole a 0 token attivate: 1. Morning-Briefing (giorni lavorativi 07:30 con archiviazione QX0ZQ e telegramma), 2. Weekly Maintenance (giornale 23:00 con audit SQLite-Backup & Health), 3. RSS & News Watchdog (scansione oraria), più 5 webbook di funzionalità attivi per l'estrazione web, messaggistica, rendering dei documenti, ricerca di feed e sync CRM.

Benefici: *Combina processi di fondo deterministici a 0 token con decisioni cognitive di agenti nei punti chiave.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

12. Vista: QX1ZQ Gateway (QX0ZQ AI Router Dashboard)

Problema: Il legame fisso con un unico fornitore di cloud porta all'arresto immediato del sistema in caso di limiti di velocità o guasti.

Meccanica: Dashboard centrale proxy (porta 20128 / 8000) con vault multi-provider, misura della latenza in tempo reale (ms), monitor di errore HTTP (429/500), breaker automatico del circuito e cascata di falda a 4 livelli (GLM -4.7 Flash -> Gemini 2.5 Flash -> Claude 3.5 Sonnet -> Llama locale 3.3 70B).

Benefici: *Assicura il 99,9 % di disponibilità di tutti i processi cognitivi a costi minimi di API.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

13. Visualizza: QX0ZQ (Guardian & Token Analytics in 3 Sub-Views)

Problema: Mancanza di trasparenza su quale agente, quale modello o sessione singola è responsabile della maggior parte dei token e dei costi.

Meccanica: Fornisce filtri temporali (24h, 7d, 30d, totale), timer inattivi e 3 sub-views: 1. descrizione e grafico di tendenza giornaliera con tabelle di profilo/modello e modulo di protezione-braccia, 2. ispettore di consumo di sessione singola (input, output, cache-read token selezionabili), 3. drossamento delle risorse e delle task (regolatore di scorrimento 1 ..16 Tasks).

Benefici: *Controllo completo dei costi, ad eccezione del singolo ID di sessione, incluso il limite immediato di bilancio e di parallelismo.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

14. Visualizza: Estensione e modulo centrale (34 plugin Evey Speciali)

Problema: Le funzioni di sicurezza, di archiviazione e di analisi non sono ancora state attivate o disattivate in modo modulare per settore.

Meccanica: Controlla 34 plugin Hermes/Evey specializzati su 8 schedari specializzati: Sicurezza & Custode (4), Performance & Cache (2), Monitoraggio & Analytics (3), intelligenza degli agenti (6), Pianificazione & Tasks (2), ponti & interfacce (4), Memoria & Conoscenza (3), Strumenti & Execution (3) più finanza/qualità/manutenzione. Include il sistema di plugin standard & runtime hook: struttura delle cartelle modulare (plugins/< plugin_id >/ plugin.json), 4 hook di Lifecycle (on_load , on_enable , on_disable , on_unload) e 2 intercettori di strumenti (before_tool_execute , after_tool_execute) con plugin di starter preinstallati (System Metrics, Security Audit Hook, Cost Guard).

Benefici: *Consente di attivare e disattivare con precisione le singole funzionalità di sistema durante il funzionamento con un clic del mouse.* Consente estensioni runtime isolate e l'intercezione degli strumenti globali senza intervento nel codice del kernel.

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

15. Visualizzazione: QX0ZQ (Triangolo di ricerca e di evidenza)

Problema: Le semplici ricerche forniscono singoli colpi non verificati senza compensazione o valutazione della fiducia.

Meccanica: Cockpit di ricerca multilivello con profondità di ricerca selezionabile, monitor sorgente, punteggio di evidenza, protocollo di sintesi dal vivo per triangolazione di dichiarazioni contraddittorie ed esportazione a 1 click nel vault QX0ZQ. Comprende il ciclo di ricerca e riflessione subacqueo v2 (Re-Search Loop): la separazione di ricerche complesse in 3 .05 domande parziali, analisi autonoma delle lacune, fino a 3 procedure di ricerca

iterative successive, Fallback Multi-Provider (SearXNG / Jina Reader) e protezione SSRF.

Benefici: *Produce dossier di mercato, legali e tecnici resistenti e riproducibili.* Completa le lacune di conoscenza in modo autonomo attraverso ricerche ricorsive successive a fonti contraddittorie o incomplete.

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

16. Visualizza: Telemetria & Logs (3-fonti Live Audit Stream)

Problema: In caso di errori nei container o nei prompt degli agenti, è stato necessario cercare con fatica nel terminale tramite i comandi Docker.

Meccanica: Include 3 fonti di log in tempo reale: Logs di container fisici docker (stdout/stderr degli 11 container), log cognitivi degli agenti e il trail di audit crittografico. Con filtri di livello (DEBUG , INFO , WARN , ERROR , Auto-Scroll-Lock ed esportazione.

Benefici: *Registratori di volo incorruttibili per la diagnosi di guasti e la verifica della conformità.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

17) Studio speciale: Arcon Prompt Studio (832 modelli & Fabric QX0ZQ)

Problema: le attività formulate in modo impreciso portano a allucinazioni, formati di uscita errati e costosi rettifica.

Meccanica: scatola integrata con 824 modelli Enterprise in 16 categorie professionali, oltre 255 modelli Fabric SOP e la formula a 5 campi (1a bobina, 2a cornice, 3a funzione, 4a formattazione, 5a regola di vendita/negativa) e ottimizzazione di IA prompt.

Vantaggi: Porta ogni delegazione in una struttura trasparente e la trasmette cliccando su chat o missione di Kanban.

Prezzo & lato ombra (trade-off): come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (richiesta di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se vengono eseguite azioni LLM attive all'interno del modulo).

18. Studio speciale: Visual CSS Live Studio & In-Place Transform Gizmo (Alt + S)

Problema: Le regolazioni di progettazione richiedevano la ricerca di file CSS e il caricamento continuo del browser.

Meccanica: Studio di progettazione in tempo reale in movimento (Alt + S) con ispettore DOM a croce di filo, Gizmo transforme interattivo (disegnare posizione, scalare 4 angoli/angolo, disegnare a angolo), Gestore di profilo a tema e esportazione diretta CSS verso custom_studio .css.

Benefici: *Consente lo styling live esatto di pixel dell'intera interfaccia utente in esecuzione senza reload di pagine.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

19. Studio speciale: Design System Lab, AppShell Architetto & 15 componenti Studio

Problema: Le altezze, le distanze o i contrasti di intestazione differenti tra la modalità luminosa e quella oscura interferiscono con la calma visiva.

Meccanica: Include l'architetto interattivo AppShell (56px header fuggitivo, larghezza Sidebar, padding per viewport con simulazione di miniatura dal vivo) e lo studio di design di 15 componenti (da os-workspace a os-badge) con motore a doppia tema (Light/Dark).

Benefici: *Garantisce un'immagine unica e priva di abbagliamento B2B SaaS in tutte le 20 viste.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

20. Visualizza Special: QX0ZQ (11 Autunno, Live-Dock) & Globale Omnibox (Ctrl + K)
Problema: le impostazioni erano sparse; la navigazione tra 20 viste e 70 agenti richiedeva molti clic del mouse.
Meccanica: QX0ZQ raggruppa 7 scheda amministrativa (QX1ZQ incl. YAML -editor, container e servizi, provider & gateway, metodologie, plugin, comunicazioni, design) e può essere agganciata a destra con 'dock dal vivo'. Omnibox (Strg + K) naviga immediatamente a qualsiasi scheda (#), agente (@) o comando. Integra il sistema di aggiornamento & Continuous Deployment 1 click: Sinc GitHub automatico con istantanee pre-aggiornamento di tutti i database SQLite (WAL) in data/snapshots/ prima di ogni pull Git, 1 clic Rollback e test minimi di stato Git contro i tempi di mount.
Vantaggi: Velocità massima di utilizzo per amministratori e utenti di potenza. Consente aggiornamenti software senza rischi con il rollback automatico del database in caso di errore.
Prezzo & lato ombra (trade-off): come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (richiesta di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se vengono eseguite azioni LLM attive all'interno del modulo).

3. ecosistema Token-Spar & FinOps (riduzione dei costi)

21. 3 animale Caps automatico costi di routing (LOW / MEDIUM / HIGH)

Problema: Quando i compiti di routine sono semplici su costosi modelli di punta (come Claude Sonnet o GPT -4o), i costi di giorno esplodono.

Meccanica: Massimali di prezzo configurabili per 1 milione di token nel QX0ZQ: LOW (max. $0.10/1M -> impone Free-Tier/Flash-Lite), MEDIUM (max. $1.00/1M -> Haiku/Gemini Flash), HIGH (No Cap -> migliori modelli di reasoning per architettura complessa).

Benefici: *Riduce fino all'85% le spese API correnti nella vita quotidiana senza perdere la qualità in caso di compiti difficili.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

22. FreeLLM- API Docker-Proxy & 'auto/best-free' Routing (porta 3002)

Problema: I contingenti gratuiti di diversi fornitori sono difficili da monitorare manualmente e cambiano spesso.

Meccanica: Un contenitore dedicato per docker (freellmapi su porta 3002) raggruppa gli endpoint LLM gratuiti. Il profilo routing 'auto/best-free' sceglie automaticamente il modello a zero euro più potente disponibile.

Benefici: *Consente il funzionamento continuo degli agenti di background e triage a costi esattamente $0.00 API.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

23. Controllo del bilancio di resa senza livelli nel master chat & router (0-16k tokens)

Problema: I modelli di reasonement (thinking mode) spesso consumano migliaia di token di pensiero invisibili per le questioni banali.

Meccanica: Regolatore a 6 fasi (da 0 tok, minimo: ~1k, basso: ~2k, medio: ~4k, alto: ~8k, massimo: ~16k) nella chat footer e in agent_router .py.

Benefici: *Spegne completamente i costosi cicli di pensiero per compiti semplici e riserva un profondo reasoning per i veri casi complessi.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

24. Evey-Cache & Cache- TTL (24 ore Cache di risposta semantica & accurata)

Problema: Le domande ricorrenti identiche o molto simili hanno portato a nuove chiamate API a pagamento.

Meccanica: Il plugin evey cache memorizza le risposte dei modelli con durata di vita configurabile (standard: 24 ore TTL) e mostra separatamente i token cache read salvati nell'ispettore di sessione.

Benefici: *Risponde alle domande ripetute in 0 millisecondi a 0 costi di token.*

Prezzo & lato ombra (trade-off): Se la cache- TTL è troppo alta (> 72h), il sistema fornisce risposte obsolete su temi dinamici.

Spegni quando? & Soglie: Disattivare le richieste in tempo reale (lezioni in diretta, stato del server). ht- TTL: 24h.

25. Max-Context & Max-History Hard-Clipper nel Guardian (32k / 20 Turns)

Problema: In ogni nuova domanda, le sessioni di chat in continua crescita trasmettono nuovamente l'intera storia e fanno salire i token in entrata in modo esponenziale.

Meccanica: Barriera di accesso regolabile per la finestra di contesto massima (ad esempio 32.000 token) e il numero massimo di messaggi di chat trasmessi (ad esempio, gli ultimi 20 turni).

Benefici: *Ricompensa in modo affidabile il massimo possibile costo di input per turno.*

Prezzo & lato ombra (trade-off): Taglia i messaggi sopra i token 32k o 20 giri duri, il che elimina il contesto molto prima.

Spegni quando? & Soglie: Per analisi di documenti profonde, aumentare a 64k .

26. Clipper di token dinamico per modelli a basso contenuto di testo (backend/ token_clipper .py)

Problema: Modelli compatti o locali (8k/16k) si spengono quando grandi edizioni di strumenti si spengono la finestra di contesto.

Meccanica: Se obbligatoriamente mantiene il prompt del sistema (indice 0) e l'ultima istruzione dell'utente, accorcia i blocchi singoli troppo a lungo in modo intelligente e rimuove le fasi intermedie più vecchie.

Benefici: *Consente l'uso stabile di piccoli modelli estremamente economici anche in un'ampia gamma di compiti.*

Prezzo & lato ombra (trade-off): Riduce le spese degli strumenti a metà ([... SNIP...]), che possono far mancare singole righe di codice nel testo grezzo.

Spegni quando? & Soglie: Disabilita a breve termine quando sono necessari i log completi grezzi.

27. Sliding Window Memory Compression (contesto decisionale consolidato)

Problema: Se si tratta di una semplice incisione di vecchi messaggi, le decisioni di progetto precedenti vengono perse.

Meccanica: Ricostruisce i vecchi circuiti di conversazione in un sommario compatto e li inietta come blocco [KONSOLIDIERTER PARLARLATO & ENTSCHEIDUNGSKONTEXT] alla posizione 1.

Benefici: *Risparmia fino al 70% di token di conversazione storica, se riceve tutte le decisioni prese.*

Prezzo & lato ombra (trade-off): Il passo di compressione costa ~800 token una volta e leviga le formulazioni letterali in precedenza.

Spegni quando? & Soglie: Regolare finemente, se necessario, il protect_last_n (ore 20) nel QX0ZQ.

28. LRU -K Page Swapping per subagente- RAM (backend/ context_swapper .py)

Problema: I subagente paralleli detengono enormi quantità di dati grezzi nello stesso tempo nella memoria attiva di prompt.

Meccanica: Risparmia le pagine di contesto necessarie a partire dall'80% di utilizzo in base al principio di recupero più basso in SQLite e le ricarica solo se necessario (riidratazione).

Benefici: *Resiste alle prompt magrissime e impedisce inutili tasse token per conoscenze di fondo inattive.*

Prezzo & lato ombra (trade-off): Il trasferimento e il caricamento di agenti RAM sul disco genera ~10 .25 ms I/O-latenza durante il cambiamento di contesto.

Spegni quando? & Soglie: Lasciare attivo il carico RAM dall'80% per evitare le cadute di OOM.

29. JEV Router . py Prefiltrazione deterministica dell'intent (jev_router .

Problema: Ogni semplice istruzione di sistema su un grande modello di lingua per classificare, spreca token e tempo.

Meccanica: Classifica i dati in entrata prima della chiamata LLM con l'esecuzione di QX0ZQ Vector e l'euristica di

parole chiave.

Benefici: *Passa comandi di sistema chiari direttamente alla routine di destinazione senza usare un singolo tasto LLM.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

30. ASD - STE100 Simplified Technical English Prompter (.agents/skills/asd-ste100-prompter)

Problema: I prosatexti di massa nei prompt di sistema e nelle istruzioni degli agenti non sono necessari per ogni singola chiamata API.

Meccanica: Trasforma le istruzioni secondo lo standard aereo ASD - STE100 (un significato per parola, verbi attivi, frasi corte, zero parole di riempimento).

Benefici: *Riduce la lunghezza dei token dei prompt di sistema di circa 30 ..40 % e riduce al contempo la velocità di errore nelle chiamate degli strumenti.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

31. Compressori anti-AI-Slop & output (. agents/ skills/ anti-ai-slop)

Problema: I modelli linguistici producono per impostazione predefinita lunghe fessure di cortesia, introduzioni e ridondanti sommari (token di output teure).

Meccanica: Interrompe le correnti di IA, le frasi vuote e l'arrotolamento artificiale direttamente nel prompt del sistema e ripulisce le edizioni.

Benefici: *Risparmia fino al 25% dei token di output particolarmente costosi e fornisce testi specializzati utilizzabili immediatamente.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

32. Pre-Flight stima dei costi e delle previsioni di bilancio (backend/ cost_estimator .py)

Problema: Prima di iniziare grandi compiti non era prevedibile quanto costerebbe la corsa.

Meccanica: Con POST /api/router/estimate-cost calcola in anticipo i token in entrata/uscita previsti e l'importo in dollari sulla base dell'attuale listino prezzi del modello e li bilancia con il bilancio task.

Benefici: *Avvertite di non correre prima che il primo centesimo venga speso.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

33. Bilancio dinamico per task con downgrade & 100% di risparmio 80%

Problema: Una singola operazione complessa ha potuto consumare inosservato tutto il budget giornaliero.

Meccanica: Controlla i costi cumulativi per attività (ad esempio limite $0.25): automaticamente riduce l'80% a un modello di risparmio economico e trattiene il lavoro al 100% con richiesta di condivisione di telegramma.

Benefici: *Impedisce agli utenti di effettuare ricerche autonome a più livelli.*

Prezzo & lato ombra (trade-off): Con il raggiungimento dell'80% del bilancio del Task, viene abbassato su un modello di risparmio più economico; il 100% viene utilizzato per la chiusura dell'hard-stop.

Spegni quando? & Soglie: Task budget (ora: 0,50 $) per le grandi gare architettoniche alzare a $ 2,00.

34. Limitazione Loop-killer & Max-steps (15 passi / Jaccard > 90%)

Problema: Se un agente si mette in un ciclo continuo in caso di chiamata di uno strumento difettoso, brucia i crediti in pochi secondi.

Meccanica: Controllate ogni passo per verificare le ripetizioni esatte (>= 3) o le coppe circolari semantiche (> 90 % di somiglianza) e scattate le attività al più tardi dopo il limite Max Steps configurabile (predefinito: 15 passi).

Benefici: *Protegge il conto da cicli senza fine inosservati durante la notte.*

Prezzo & lato ombra (trade-off): I limiti troppo severi (Max Identisch = 2 o Jaccard < 0.80) fermano in anticipo le attività iterative legittime, come la correzione graduale dei test o la paginazione.

Spegni quando? & Soglie: Spegnere a breve termine se un agente deve intenzionalmente lavorare molti passaggi di rettifica uniforme (ad esempio, migrazione di massa di 50 file). . Soglie: Max Identisch: 2 = ferma Retries . . > 5 = rettifica costosa (h: 3). Jaccard: < 0.80 = falsi allarmi . . .96 = rileva i cicli in ritardo (h: 0.90). Max Passi: h 15.

4. Kernel, architettura Scheduling & Daemon (Pillar 1)

35° Hermes Master Orchestrator (Hermes-Core)
Problema: i singoli agenti non coordinati lavorano l'uno contro l'altro o perdono di vista l'obiettivo generale.
Meccanica: centrale di processo per l'analisi degli ordini, selezione dei ruoli, monitoraggio e riduzione dei risultati.
Benefici: assicura la coerenza strategica tra tutti i 70 specialisti.
Prezzo & lato ombra (trade-off): Più servizi di sfondo attivi e alta resa ('high') aumentano il carico di base e i costi dei token.
Spegni quando? & Soglie: controllabile tramite le 14 sezioni della scheda HERMES CORE 'granular.

36. Tickless Kernel Daemon & Asyncio Event-Loop (porta 8420 / 8501)

Problema: Polling permanente a ritmo di secondo spreca risorse CPU e energia del sistema.

Meccanica: Calcola l' esatto momento esatto del prossimo 'Sleep-until-next-due' e in questo modo il kernel viene messo in stato di riposo.

Benefici: *Carico minimo del sistema al minimo per la risposta senza ritardi.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

37. Preparazione prioritaria Scheduler (P0 .P3 coda)

Problema: Utenti urgenti nella chat diretta hanno dovuto aspettare quando i lavori di sfondo hanno occupato tutte le slot.

Meccanica: Serpente prioritario: compiti P3 (direttamente chat / emergenza) superare immediatamente i lavori di routine P1/P2 in esecuzione.

Benefici: *Garantisce tempi di risposta rapidi nella finestra di dialogo nonostante un carico di sfondo elevato.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

38. Drosseggio di concurrency libero scalabile (backend/ concurrency_throttle .py)

Problema: Troppi agenti simultanei portano a limiti HTTP -429, e punte CPU 99 %.

Meccanica: Regola i lavoratori attivi tramite un semaforo Asyncio (regolabile con slider da 1 a 16 task paralleli nel Guardian e Control Center).

Benefici: *Consente la protezione ridotta su computer portatili e la gestione ad alto carico su stazioni di lavoro.*

Prezzo & lato ombra (trade-off): I limiti di stagnazione a bassa velocità (< 30 secondi) si annullano erroneamente i modelli di reasoning lento (ad esempio, o3/R1) nel mezzo del processo di pensiero; più lavoratori aumentano il carico di punta RAM e API.

Spegni quando? & Soglie: Spegnere l'hardware in caso di analisi individuali estremamente lunghe o di esecuzione locale di LLM su hardware lento. . Soglie: Max Worker: 1 .62 = Riposo . . . . . .

39. watchdog e recupero autonomo a 30 secondi (watchdog_daemon .py)

Problema: Le tasks bloccate o congelate sono rimaste permanentemente bloccate nello stato in_progress.

Meccanica: Controlla tutti i processi in esecuzione ogni 30 secondi, interrompe i task zombie in timeout (120s) e trasferisce le attività orfane nel recupero automatico di crash.

Benefici: *Tiene tutte le slot da lavoro senza parti di carretti.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

40. Supervisor deadlock & stagnazione (backend/ deadlock_supervisor .py)

Problema: Le dipendenze reciproche tra due agenti possono portare ad un congelamento silenzioso.

Meccanica: Rilevare gli stati di attesa circolare e le task senza progressi (> 45 secondi) e dissolvere il blocco in modo autonomo con backoff esponenziale.

Benefici: *Previene gli arresti di condotte in operazioni complesse di gruppo.*

Prezzo & lato ombra (trade-off): Un limite di stagnazione troppo basso (< 30s) interrompe i modelli di reasoning lento (o3/R1) nel bel mezzo del processo di pensiero.

Spegni quando? & Soglie: Quando si utilizzano modelli di resa, aumentare il limite di stagnazione a 90-120s (ore 45).

41. Autoguarigione della flotta e ripristino dell'auto-container (backend/ fleet_self_healing.py)

Problema: Se uno degli 11 contenitori Docker non funziona, è stato necessario riavviarlo manualmente.

Meccanica: Controlla tutti i contenitori Core tramite il docker-socket-proxy protetto e avvia autonomamente un nuovo avvio, incluso l'ingresso di audit, in caso di guasto (> 60s).

Benefici: *Assicuratevi di gestire il 24/7 incustodito dell'intero paesaggio Docker.*

Prezzo & lato ombra (trade-off): Un contenitore danneggiato in modo permanente può essere in un ciclo di riavvio a causa del riavvio automatico.

Spegni quando? & Soglie: Durante la manutenzione manuale dei container, disattivare per un breve periodo.

42. Standup e sveglia proattiva del mattino (backend/ autonomous_standup .py)

Problema: La mattina mancava una panoramica compatta delle corse notturne, dei costi e delle decisioni da prendere.

Meccanica: Genera ogni giorno alle 08:00 un rapporto consolidato sullo stato del QX0ZQ Vault (01_PROGETTO/Daily_Briefings/) e lo pompa a Dashboard e Telegram.

Benefici: *Inizia ogni giorno lavorativo con un progetto di decisione finale.*

Prezzo & lato ombra (trade-off): Ogni mattina automatico di bliefing consuma ~1.500 .3.000 gettoni al momento impostato.

Spegni quando? & Soglie: Spegni se non è richiesta una relazione giornaliera del mattino (ore 08:00).

5. Modello di processo multi-agente, gerarchia e sintesi (Pillar 2)

43. 70 Profilo degli agenti dichiarati e file del personale di Markdown
Problema: Gli assistenti generici di IA non hanno coscienza del ruolo e non hanno standard qualitativi definiti.
Meccanica: ognuno dei 70 specialisti è ancorato a agent_profiles .json e a un proprio fascicolo personale (00_HR_agente/) con prompt di sistema, percorso modello, utensile e SOP.
Benefici: crea personale virtuale affidabile per la strategia, la legge, la costruzione, le finanze, il marketing e l'IT.
Prezzo & lato ombra (trade-off): come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (richiesta di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se vengono eseguite azioni LLM attive all'interno del modulo).

44. Robert Delegazione, Sincronizzazione dei fan-in e Lisa Contrattuale Marker (SYNTHESE - BEREIT)

Problema: I risultati parziali sono stati persi o sono stati raggruppati in modo incompleto per i compiti parziali paralleli.

Meccanica: Robert smonta grandi progetti; un blocco di processo (_MC_DATA_LOCK) attende il completamento di tutti i compiti parziali (fan-in). Lisa raggruppa i rapporti sotto esame del marcatore obbligatorio SYNTHESE - BEREIT; Robert rilascia il FREIGABE formale nella riga 1 .

Benefici: *Garanzia di qualità multilivello, sulla base di un'autentica governance aziendale.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

45. Desk palmare (backend/ handoff_desk .py)

Problema: Silenziosi perdite di contesto quando si consegna da uno specialista al successivo.

Meccanica: Genera i registri di consegna formali con ipotesi aperte, deliverabili e SHA -256 che devono essere autenticati dall'agente ricevente.

Benefici: *Interfacce sicure di verifica tra tutti i reparti.*

Prezzo & lato ombra (trade-off): I test Strikte SLA respingono risposte molto brevi ma corrette quando min_content_length è troppo alto (> 300).

Spegni quando? & Soglie: Standard: 100 caratteri Lunghezza minima, 60 min. Turnaround.

46. Schedulatore DAG a due fasi per l'esecuzione parallela del progetto (backend/ dag_scheduler .py)

Problema: La gestione periodica di molti compiti parziali richiede troppo tempo per i grandi progetti.

Meccanica: Calcola il grafico aciclico mirato (DAG): esegue ricerche indipendenti (livello 1) in parallelo e avvia automaticamente la sintesi (livello 2).

Benefici: *Abbrevia di un multiplo la durata di analisi complesse.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

47. Self-Correction & Tool Execution Reflection Loop

Problema: Gli errori di sintassi o parametri in fuga nelle chiamate degli strumenti hanno portato all'interruzione immediata.

Meccanica: Se gli errori dello strumento si cancellano, il messaggio di errore riflette e corregge i parametri di chiamata in modo autonomo in un massimo di 3 tentativi.

Benefici: *Maggiore robustezza nelle operazioni di file, database e web.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

48. Flussi di lavoro a lungo termine interrompibili con snapshotting di Stato SQLite

Problema: Se un'operazione aspetta per ore una condivisione utente, occupa una memoria inutile o perde la sua condizione quando si riavvia.

Meccanica: Congela lo stato di esecuzione esatto nella tabella workflow_checkpoints e riprende il lavoro senza problemi dopo la condivisione.

Benefici: *Consente processi aziendali di più giorni durante il riavvio.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

49. Dual-Agent Consenso-Gate & Adversarial Critic Review (backend/ consensus_gate .py)

Problema: Un singolo agente non nota spesso i propri errori mentali o allucinazioni in compiti critici.

Meccanica: In caso di interventi P3 e di interventi critici, la contro-lettura formale è richiesta da un agente indipendente di Revisor (Red-Team/Critic). Nella scheda Kanban, il gate di qualità a 4 occhi automatizzato (Katharina Völk / Robert Review con punteggio 1 .0100% e lock di stato in caso di necessità di revisione).

Benefici: *Principio di quattro occhi per tutti i risultati critici per l'impresa.*

Prezzo & lato ombra (trade-off): Triplica il consumo dei token (3x) e aumenta il tempo di risposta di +6 a +20 secondi; l'unanimità comporta il rischio di Deadlock.

Spegni quando? & Soglie: Attiva solo dalla priorità P3/P4 e seleziona la maggioranza di 2/3.

6. Driver MCP, Servizi esterni e Strumenti multimodali (Pillar 3)

ecosistema Arcon QX1ZQ & QX0ZQ (Porte 3005, 8090, 8092)

Problema: I contatti con i clienti, le storie di posta elettronica e gli assist multimediali erano separati dagli agenti dell'IA.

Meccanica: Frontend dedicato CRM (porta 3005) con backend QX0ZQ (porta 8090) e server multimediale (porta 8092), completamente collegato tramite scripts/ archon_crm_mcp .py.

Benefici: *Gli agenti possono richiamare, rintracciare, valutare (scoring di comando) e assegnare corrispondenza.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

51. Matching automatico di posta elettronica e Archivio LinkedIn

Problema: Migliaia di contatti provenienti da esportazioni LinkedIn e caselle postali per posta elettronica sono stati confrontati manualmente.

Meccanica: Sincronizza le caselle postali (-fetch-mail), assegna automaticamente le mail ai contatti CRM esistenti e importa i CSV completi di collegamento LinkedIn, inclusa la deduplicazione.

Benefici: *Costruisce un database completo e arricchito di relazioni in minuti.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

52. Robert Tool-Permission Manager (backend/ tool_permission_manager .py)

Problema: Gli agenti non devono interrompere in modo inefficace i pacchetti Python/Node mancanti, ma non devono installare nulla senza necessità.

Meccanica: Robert presenta una richiesta di installazione formale (pending_permission) con comando e motivazione; solo dopo la condivisione di 1 clic il pacchetto verrà installato.

Benefici: *Espandibilità controllata con protezione completa del sistema host.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

53. Event-Driven Webhook Mesh (backend/ event_mesh .py)

Problema: Gli eventi esterni di GitHub, moduli o sistemi terzi non hanno potuto attivare azioni di agenti.

Meccanica: Ricevitore webhook universale che convalida i payloads esterni JSON e li converte direttamente in task eseguibili Kanban.

Benefici: *Collega QX0ZQ in tempo reale al resto del paesaggio IT.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

54. Multi-channel Notification Hub centrale (backend/ notification_hub .py)

Problema: Gli avvisi importanti sono stati cancellati nel QX0ZQ, mentre i messaggi non importanti sul telefono sono stati disturbati.

Meccanica: Itinerario messaggi rigorosamente secondo la gravità: INFO -> Toast UI, WARNING -> Dashboard-Log, CRITICAL -> Push telegram immediato.

Benefici: *Allarme preciso senza preavviso.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

55. Pipeline multimodale Vision & Screen Capture (backend/ vision_pipeline .py)

Problema: Problemi visivi o tabelle nelle immagini hanno dovuto essere descritti con fatica.

Meccanica: Registrazione diretta dello schermo nel browser (button della fotocamera nella chat) e analisi automatica OCR e strutturale per le immagini caricate (.png, .jpg, .webp).

Benefici: *Gli agenti vedono esattamente quello che vedi sullo schermo.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

56. Apollo Voice Engine . Edge- TTS & Whisper Diktat (apollo_voice_engine .py)

Problema: Inserire testi lunghi sullo smartphone o sul desktop richiede tempo; i rapporti sono necessari per ascoltare i messaggi durante il viaggio.

Meccanica: Pipeline bidirezionale: dittato microfono (speech-to-text) direttamente nel campo di inserimento chat e pronuncia naturale (testo-to-speech) per i briefing. Integrazione vocale telegram completa (POST /api/telegram/voice-relay e POST /api/voice/synthesize) per i memo audio mobili via edge-tts (de-DE-ConradNeural).

Benefici: *Trasforma QX0ZQ in un assistente esecutivo a comando vocale.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

57. Oracle News Radar & SEO Execution Agent

Problema: Le variazioni di mercato e le vulnerabilità SEO dei propri siti web sono state verificate solo sporadicamente.

Meccanica: Oracle News Radar analizza continuamente fonti specializzate per valutare l'importanza del progetto; l'agente SEO controlla le pagine su meta tag, struttura e densità di parole chiave.

Benefici: *Ottimizzazione continua del mercato e della visibilità in background.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

7. Memoria, igiene del database & QX0ZQ 2nd Brain (Pillar 4)

58. Layer centrale SQLite WAL -Memory (5 tabelle standard)

Problema: L'accesso simultaneo alla lettura e scrittura di più agenti ha causato conflitti nei file semplici.

Meccanica: Database di negoziazione SQLite (data/ agentic_memory .db) in modalità di logging write-ahead (WAL) con le 5 tabelle principali tasks, conversations, learnings, kv_store e user_profile .

Benefici: *Accesso parallelo senza blocco con massima sicurezza dei dati.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel

dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

59. Sidecar Kanban a prova di processo con scambio di file atomico (backend/ kanban_sidecar .py)

Problema: I metadati come i limiti di WIP o i tag potrebbero corrompere con le operazioni di scrittura parallele.

Meccanica: Protegge config/ kanban_meta .json tramite un mutex di processo e operazioni di riplace write nucleari.

Benefici: *Previene la perdita di dati in caso di aggiornamenti ad alta frequenza.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

60. Docker- RAG -Bridge a QX0ZQ (5.068 documenti indicizzati sulla porta 3001)

Problema: Le conoscenze disponibili nel sistema di memoria del vettore Docker non sono state utilizzate automaticamente nelle normali richieste di chat.

Meccanica: backend/ rag_bridge .py collega il nucleo dell'agente direttamente al Workspace QX0ZQ ARCHON e fornisce le frasi di testo pertinenti come context grounded.

Benefici: *Risposte basate sui tuoi documenti aziendali reali invece di stime generali.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

61. Disposition di cartella di progetto in tre fasi nel vault QX0ZQ

Problema: Quando gli agenti creano file, senza regole fisse, si crea rapidamente confusione nel Vault.

Meccanica: Per ogni progetto, imposta automaticamente la struttura a 3 fasi: 01_Report intermedi, 02_Media e 03_Prodotti finali.

Benefici: *Separazione pulita tra i progetti grezzi e i risultati finali ottenuti.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

62. QX0ZQ Vault Indexer & Map automatica di contenuti (backend/ vault_hygiene .py)

Problema: Le nuove relazioni del progetto sono rimaste spesso senza collegamento nel grafico QX0ZQ ('Orphan Notes').

Meccanica: Scansiona il Vault, rileva le note orfane e aggiorna automaticamente il _MOC_Project_Index.md centrale con Wikilinks.

Benefici: *In ogni momento mantiene completamente connesso il grafico della conoscenza in QX0ZQ.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

63. Deduplicazione autonoma Fact-Harvesting & 85% di uniformità (backend/ post_task_pipeline .py)

Problema: Le principali conclusioni tratte dai compiti completati sono state perdute o sono state archiviate più volte in doppio.

Meccanica: Dopo la conclusione di un'attività, automaticamente estrae nuovi fatti per learnings e user_profile e impedisce dubletten a partire dall'85% di somiglianza di testo.

Benefici: *Il sistema diventa più intelligente con ogni compito fatto senza dover scaricare il deposito.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

64. Backup automatico DB online con rotazione di 7 giorni (backend/ db_backup_manager .py)

Problema: Senza una banca dati regolare, i difetti hardware rischiano di perdere tutte le attività e i profili.

Meccanica: Esegue PRAGMA wal_checkpoint (TRUNCATE) in esecuzione e salva archivi compressi .db.gz in .hermes/backups/db/.

Benefici: *Il backup dei dati senza lacune senza un secondo di tempo di interruzione.*

Prezzo & lato ombra (trade-off): I backup completi giornalieri occupano ~50 .500 MB di memoria del disco rigido a seconda della durata di conservazione.

Spegni quando? & Soglie: Standard: 7 giorni di conservazione (db_backup_keep_days).

65. Ottimizzatore autonomo SQLite Vacuum & Frammentazione (backend/ db_optimizer .py)

Problema: I record cancellati e gli aggiornamenti frequenti gonfiano inutilmente i file SQLite per mesi.

Meccanica: Esegue l'ottimizzazione e l'ottimizzazione VACUUM su tutti i database PRAGMA tramite POST /api/system/optimize-db e calcola lo spazio di archiviazione condiviso.

Benefici: *I tempi di consultazione sono mantenuti in modo permanente nell'area millisecondi.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

66. Rollback atomico pre-write & 1 click (backend/ snapshot_manager .py)

Problema: Se un agente sovrascrive un file in modo difettoso, il ripristino del pre-stato era costoso.

Meccanica: Prima di ogni accesso alla scrittura, salva automaticamente l'originale in .hermes/snapshots/ e ripristinalo immediatamente cliccando o QX0ZQ. Inoltre, come snapshot pre-aggiornamento di tutti i database SQLite prima pull GitHub e aggiornamenti container integrato in data/snapshots/.

Benefici: *Rete di sicurezza completa in tutte le modifiche di file e codice.*

Prezzo & lato ombra (trade-off): Ogni snapshot pre-write costa ~5 .15 ms Disk-I/O prima di sovrascrivere un file.

Spegni quando? & Soglie: Blocco anello limitato a 50 snapshot per impostazione predefinita.

67. Anteprima Git compatibile dal vivo diff (backend/ diff_engine .py)

Problema: Prima di condividere, si desidera vedere esattamente quali righe cambiano invece di rileggere l'intero file.

Meccanica: Crea un confronto di righe colorate (rosso = rimosso, verde = nuovo) tra il progetto di stock e l'agente.

Benefici: *Verificare rapidamente le modifiche di codice e documento nel gate HITL.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

8. Sicurezza, Zero-Trust Governance & System Diagnosi (Pillar 6)

68. Log di audit QX0ZQ concatenato criptograficamente (backend/ audit_log .py)

Problema: I log di testo semplici possono essere modificati o eliminati successivamente.

Meccanica: Ogni evento è legato all'hash SHA -256 della voce precedente (principio Blockchain) ed è verificabile tramite GET /api/audit/verify per manomissione.

Benefici: *Prova di verifica di tutte le operazioni di sistema e di condivisione.*

Prezzo & lato ombra (trade-off): Overhead minimo di scrittura (~2 ms per voce); la catena crittografica cresce per mesi.

Spegni quando? & Soglie: Lasciare acceso in modo permanente per una governance sicura.

69. Local Security Guard . . 25 Firmature Prompt-Injection & Jailbreak

Problema: Le email o le pagine web malvage possono contenere comandi nascosti ('Ignore previous instructions...').

Meccanica: Scansiona ogni ingresso di testo esterno con 25 schemi di attacco prima di entrare nel contesto di un

agente.

Benefici: *Resiste alle injezioni indirette di prompt al confine del sistema.*

Prezzo & lato ombra (trade-off): L'euristica molto severa può causare falsi allarmi (falsi positivi) nei testi di sicurezza IT o nei prompt di testing.

Spegni quando? & Soglie: Solo per i test di Red Teaming autorizzati per un breve periodo di tempo.

70. Secret-Masking & Red-Teaming Output Guard (backend/ secret_masker .py)

Problema: Le chiavi API o le password potrebbero comparire accidentalmente nei log, nelle report o nelle edizioni di chat.

Meccanica: Filtra i testi in uscita in tempo reale su un modello di chiave (sk-.., bearer, password) e li sostituisce con [PROTETTE_KEY].

Benefici: *Impedisce la divulgazione accidentale di dati riservati di accesso.*

Prezzo & lato ombra (trade-off): La mascheratura aggressiva della regolazione può in rari casi oscurare le stringhe di base64 o hash innocue nelle edizioni di codice.

Spegni quando? & Soglie: Lasciare attivo in modo permanente durante il funzionamento produttivo per proteggere i tasti API.

71. Proxy socket docker coperto (porta 2375 internamente)

Problema: L'accesso diretto a /var/run/docker.sock dà ai container diritti di root senza restrizioni sul sistema host.

Meccanica: Attiva un contenitore proxy restrittivo che passa solo per i comandi di stato e di riavvio esplicitamente autorizzati.

Benefici: *Protegge il sistema operativo host secondo il principio Zero-Trust.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

72. Benchmarker A/B e Scorer di prestazioni Empirischer (backend/ model_benchmarker.py)

Problema: Nuovi modelli linguistici appaiono settimanalmente e spesso non sono chiari per le proprie mansioni aziendali.

Meccanica: Prova due modelli paralleli con attività identiche e confronta tempo di risposta, costi token e qualità dei risultati.

Benefici: *Base di dati obiettivi per la selezione dei modelli nella squadra degli agenti.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

73. Altro motore diagnostico a 360° di valutazione della testa (backend/ system_diagnostics .py)

Problema: Le singole metriche spesso non mostrano in un colpo d'occhio quanto sia sano il sistema generale.

Meccanica: Calcolata dallo stato della flotta Docker (35%), dalla concurrency quie (20%), dall'integrità dell'audit (20%), dalla libertà di deadlock (15%) e dalla raggiungibilità RAG (10%), un punteggio totale dello 0.0100 %, comprese le raccomandazioni d'azione.

Benefici: *Diagnosi totale istantanea del sistema operativo tramite GET /api/system/health-score.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

9. Auto-evoluzione, mercati e auto-documentazione (Pillar 7)

74. Sintesi autonoma di abilità con feedback della cartella personale (backend/ skill_synthesizer .py)

Problema: Finora, le soluzioni multilivello di successo hanno dovuto essere programmate manualmente come nuove abilità.

Meccanica: Se lo strumento di successo ricorrente riconosce, genera automaticamente un progetto SKILL .md, lo inserisce nel file personale dell'agente e lo presenta per la condivisione.

Benefici: *Il sistema amplia le sue capacità di funzionamento in modo autonomo.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

75. Mercato dinamico del talento e della skill (skill_markt.js & AgentSkillsHub)

Problema: Riscrivere nuovi ruoli o flussi di lavoro per nuovi settori di attività è un'attività che richiede tempi lunghi.

Meccanica: Catalogo ricercabile per l'importazione ed esportazione di profili di agenti completi da 1 clic e abilità esterne (compresa integrazione agentskillshub.dev).

Benefici: *Scala la flotta degli agenti in pochi secondi per i nuovi progetti.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

76. Sincronizzazione completa del fronte-end (238 fetch / 581 percorsi / 29 modale)

Problema: Nelle grandi applicazioni web, i pulsanti orfani o modal senza legami spesso portano a errori silenziosi.

Meccanica: Il gasdotto di prova automatizzato garantisce che tutte le 238 chiamate API frontend e 29 modali siano perfettamente collegate alle 368 router backend.

Benefici: *Garantito 0 collegamenti morti e 100% controlli gestibili in tutte le viste.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

77. Generatore di master e manuale autonomo (backend/ system_dossier_generator .py)

Problema: Le documentarie architettoniche tendono a diventare obsolete pochi giorni dopo la loro creazione.

Meccanica: Genera simultaneamente il fascicolo di architettura del sistema attuale nella cartella del progetto e nel vault QX0ZQ (00_DOCUMENTAZIONE/) premendo un pulsante (POST /api/system/generate-dossier).

Benefici: *Mantenere la documentazione del sistema costantemente aggiornata senza necessità di manutenzione manuale.*

Prezzo & lato ombra (trade-off): Come un'infrastruttura di sistema o di visualizzazione permanente integrata nel dashboard (necessità di base trascurabile di RAM -/ CPU; i costi di token sono sostenuti solo se le azioni LLM sono eseguite attivamente all'interno del modulo).

10. Matrice di riferimento: plugin QX0ZQ /Hermes controllabili

ID & plugin	Categoria tematica	Meccanica e funzione di base nel sistema operativo
78. evey- cost-guard	Sicurezza e sentinelle	Monitoraggio dei costi in tempo reale e chiusura di bilancio dura al giorno e al Task.
79. evey-session-guard	Sicurezza e sentinelle	Sorveglia i periodi di sessione, i limiti dei token e previene le sessioni zombie.
80. evey-email-guard	Sicurezza e sentinelle	Scansiona le email in arrivo prima del triage su phishing e injection prompt.
81. evey-watchdog	Sicurezza e sentinelle	Controlla i processi attivi nel tempo di 30s e riavvia

		automaticamente i lavoratori bloccati.
82. evey-cache	Performance & Cache	Cache di risposta 24 ore per rispondere immediatamente alle richieste ripetute gratuitamente.
83. disk-cleanup	Prestazioni e manutenzione	Pulisce manufatti temporanei, log obsoleti e rilascia la memoria del disco rigido.
84. evey-telemetry	Monitoraggio e analisi	Riceve flussi di token, lanterne e chiamate di modello per QX0ZQ e Guardian.
85. evey status	Monitoraggio e analisi	Continua ping sanitario su tutti gli 11 contenitori Docker e gateway.
86. evey-delegation-score	Monitoraggio e analisi	Valuta il tasso di successo e qualità dei trasferimenti di compiti tra agenti.
87. evey-autonomy	Intelligenza degli agenti	Controlla il grado di autonomia per le decisioni di fondo multilivello.
88. evey-countcil	Intelligenza degli agenti	la formazione di una maggioranza e di un consenso tra diversi specialisti (matermine).
89. evey-reflect	Intelligenza degli agenti	Flessione autoriflessione per la correzione automatica degli errori prima del rilascio dei risultati.
90. evey-learner	Intelligenza degli agenti	Estrae le conoscenze permanenti da compiti svolti per la tabella di learnings.
91. evey-identity	Intelligenza degli agenti	Garantisce la rigorosa conformità a SOUL .md, tonalità e guide di rotolo.
92. evey-habits	Intelligenza degli agenti	Gestisce le routine ricorrenti come la lettera di standup delle 08:00 al giorno.
93. evey-proactive	Intelligenza degli agenti	Consente agli agenti di proporre autonomamente le proprie azioni in caso di strozzature individuate.
94. evey-goals	Pianificazione e task	Collega compiti operativi specifici a Kanban con obiettivi aziendali di alto livello.
95. evey-scheduler	Pianificazione e task	Pianificazione di ordini di background, basata su tempi e priorità.
96. evey-bridge	Ponti e interfacce	Disco dati interno tra monolito FastAPI, demone Hermes e Sidecars.
97. evey-mqtt	Ponti e interfacce	Protocollo di messaggi leggero per i segnali di stato IoT e in tempo reale.
98. evey-moltbook	Ponti e interfacce	Sincronizza libri di lavoro strutturati e note di sintesi.
99. evey-telegram-ux	Ponti e interfacce	Controlla il routing dei nomi di chiamata (/robert, /avvocato) e i pulsanti interattivi di condivisione in Telegram.
100. evey-github	Ponti e interfacce	Collegamento per operazioni di repository, analisi commit e monitoraggio Issue.

101. evey-research	Strumenti e ricerche	Driver di ricerca profonda per lo scraping web e l'estrazione di fonti nel QX0ZQ .
102. evey-news	Strumenti e ricerche	Scanner di sfondo per feed di settore e notizie di mercato (oracle News Radar).
103. evey-digest	Strumenti e ricerche	Comprime vaste collezioni di documenti in sovrastrutture di gestione compatte.
104. evey-sandbox	Strumenti & Execution	Ambiente di esecuzione isolato per il codice Python e analisi generato nel QX0ZQ .
105. evey-rag	Memoria & conoscenza	Driver vettoriale di recupero per collegare QX0ZQ e archivi di conoscenza locali.
106. evey-memory-adaptive	Memoria & conoscenza	Adeguamenti dinamici della memoria a breve e lungo termine in funzione della pressione del contesto.
107. evey-memory-consolidate	Memoria & conoscenza	Cammina notturna per la fusione e la deduplicazione di elementi di memoria correlati.
108. evey-wallet	Finanze e bilanci	Gestisce API-Buona fortuna, contingenti e limiti di transazione basati sui costi.
109. evey-validate	Qualità e esecuzione	Controlla le spese per conformità allo schema, marcatore obbligatorio (SYNTHESE - BEREIT) e specifiche di formato.
110. evey-verification	Qualità e esecuzione	Controllo dei fatti e verifica delle dichiarazioni critiche prima della condivisione finale.
111. evey-delegate-model	Qualità e esecuzione	Seleziona automaticamente il modello di destinazione ottimale in termini di costi e prestazioni per i compiti parziali.

11. Architettura Enterprise & Resilienza

78. Replay & Time-Travel Debugging (backend/ time_travel_debugger .py)

Problema: Nei complessi flussi di lavoro degli agenti con molte chiamate degli strumenti, la debug ex post nel log test è poco chiara e non è possibile ripartire i vincoli di esecuzione.

Meccanica: Salva ogni fase con i comandi, i pulsanti e il token in execution_traces. Consente il replay interattivo (GET /api/traces /{ task_id }), il foro di stato dal passo N (POST /api/traces /{ task_id }/fork) e il rollback di file modificati da 1 clic.

Benefici: Analisi rapida delle cause e esperimenti senza rischi di cosa sarebbe se con prompt alternativi o modelli senza riavviare l'intero gasdotto.

Prezzo & lato ombra (trade-off): Aumenta disc I/O di ~5 ..15 ms per file e occupa spazio aggiuntivo sul disco rigido a seconda del limite di snapshot ~50 .

Spegni quando? & Soglie: Spegnimento su sistemi con memoria disco estremamente limitata o con generazioni di file di massa (ad esempio, manufatti di build). . Soglie: max snapshot: < 10 = breve storia . . > 200 = consumo di disco più elevato (h: 50). Max Trace Step: h 100.

79. Multi-Modello Eval-Matrix & A/B-B-Bater (backend/ model_benchmarker .py)

Problema: Le assegnazioni di modelli ad agenti sono spesso basate su ipotesi soggettive, in mancanza di parametri empirici.

Meccanica: Invia lo stesso prompt simultaneamente a due LLM configurati (POST /api/router/benchmark), misura

latenza, consumo di token e lunghezza di risposta e salva la storia persistente in 'model_benchmarks' (GET /api/router/benchmark/history).

Benefici: Selezione di modelli di dati per ruolo di agente (ad esempio, DeepSeek V3 per velocità/costi vs Claude 3.7 per architettura complessa).

Prezzo & lato ombra (trade-off): N modelli candidati costano N volte API -token per prova.

Spegni quando? & Soglie: Avvia solo per confronti di modelli, non in modalità continua.

80. Tri-Agent Consenso Gate & Arbitro (backend/ consensus_gate .py)

Problema: Le procedure critiche (refactoring del codice, migrazione) comportano il rischio di allucinazioni o di lacune di sicurezza nelle decisioni individuali.

Meccanica: Convoca un organo di tre persone (sistema Architect, Security Officer, QA Lead) (POST /api/consensus/council-vote). Ogni revisore vota indipendentemente; il rilascio richiede la maggioranza di 2/3. Tutte le decisioni sono archiviate in 'consensus_decisions' (GET /api/consensus/history).

Benefici: Minimizza i rischi ed elimina le allucinazioni single-point of failure in caso di modifiche del sistema.

Prezzo & lato ombra (trade-off): Triplica il consumo di token (3x) e aumenta il tempo di risposta da +6 a +20 secondi. In caso di voto all'unanimità, il tempo di decisione è minacciato (Deadlock).

Spegni quando? & Soglie: Spegni (o accendi la priorità 4 = Critica) quando la velocità e i costi bassi di API sono prioritari. . Soglie: A partire da priorità: Prio 2 = quasi ogni task costa 3x token . Prio 4 = solo task critici (h: Prio 3).

81. Auto-cura autonoma delle task-recupero con Git-Rollback (backend/ task_self_healer .py)

Problema: Errore di sintassi o importazioni di codice errate bloccano immediatamente le operazioni e richiedono l'intervento manuale dell'utente.

Meccanica: Isola la race di stack, esegue un back-rollback di file modificati in caso di errori di durata e trasmette all'agente un prompt di correzione strutturato (POST /api/healing/attempt). Fino a 3 tentativi di riparazione autonomi; recupero di log via GET /api/healing/logs.

Benefici: Massima resilienza per le versioni di codice e script con autoriparazione automatica prima dell'escalation nella scheda HITL.

Prezzo & lato ombra (trade-off): Ogni tentativo di autoguarigione costa 1 chiamata LLM supplementare (+3 - +12 secondi. Costo Latenza & Token). Un rollback Git automatico non consente di ottenere risultati parziali utili in caso di fallimento.

Spegni quando? & Soglie: Spegnere il rollback di Git se i progetti di codice non finiti devono rimanere nella directory di lavoro per il lavoro manuale. . Soglie: Max Repair Process: 1 = dà immediatamente su . . > 5 = costoso cascata di riparazione (h: 3).

82. Guarda la directory degli eventi e della posta in arrivo (backend/ event_mesh .py)

Problema: Finora è stato necessario avviare manualmente le attività tramite la dashboard o consultare il ponling IMAP con controllo temporale.

Meccanica: Evento centrale per Webhook esterni (POST /api/events/inbound /{channel}) e Watcher automatico per directory (POST /api/events/scan-directory) che trasferisce nuovi file in 'data/inbox/' direttamente in task di Kanban Triage.

Benefici: Evento e registrazione completamente automatica da sistemi esterni (GitHub, QX0ZQ , Stripe, scanner di file locali) senza interazione utente.

Prezzo & lato ombra (trade-off): Intervalli di scansione molto brevi (< 3 secondi) generano pull di file system continui; il sistema di standup giornaliero del mattino consuma ~1.500 .3.000 token per corsa.

Spegni quando? & Soglie: Spegni se la cartella locale Inbox non è attiva o se non è richiesta una segnalazione di briefing. . Soglie: Intervallo di scansione: 1 .62s = pulsanti di disco inutili . . > 30s = rilevamento file ritardato (ore 5s). Ora di standup: ore 08:00.

12. Architettura di autonomia, sandbox e governance

83. Sandboxs in fuga (backend/ ephemeral_sandbox .py)

Problema: Esecuzione del codice agente comporta il rischio di crash di sistema, rettifica infinita o modifiche non intenzionali dei file sull'host.

Meccanica: Esegue il codice in contenitori micro docker a vita breve e isolati con un limite di risorse duro (512 MB RAM , 1 CPU , 30s Timeout, nessuna rete) (POST /api/sandbox/execute). La scatola di sabbia viene eliminata senza residui dopo il completamento.

Benefici: Sicurezza assoluta e stabilità del sistema durante l'esecuzione e la prova automatizzata del codice.

Prezzo & lato ombra (trade-off): Container-Kalstart costa ~300 .800 ms Latenza supplementare per ogni chiamata strumento. Se la rete è disabilitata, battere pip install o le chiamate esterne API nello script non riesce immediatamente.

Spegni quando? & Soglie: Disattivare (o attivare la rete) quando gli script locali affidabili richiedono l'accesso diretto ad host o l'installazione rapida di pacchetti. . Soglie: RAM -Limit: < 256 MB = OOM -Kill in Pandas/NumPy . . > 2048 MB = Rischio di host RAM (h: 512 MB). Timeout: < 10s = Cancella build (h: 30s).

84. Consolidamento della memoria notturna (backend/ memory_consolidator .py)

Problema: Tracce in continua crescita, heartbeats e dati di dialogo obsoleti rallentano le richieste di database in funzionamento continuo.

Meccanica: Prunt in fasi di riposo (standard 03:00) Tracce obsolete e log acritici utilizzando i fres di ritenzione configurabili (settings.json). Completamente controllabile tramite le impostazioni QX0ZQ (orario, giorni di ritenzione, profondità Pruning, pulsante di run manuale).

Benefici: Banca dati permanenti e sottili, lanterne ridotte e igienizzazioni automatiche senza manutenzione manuale.

Prezzo & lato ombra (trade-off): Durante il processo di consolidamento (~10.030 sec.) si forma un carico corto di CPU/Disk; una soglia di Pruning troppo bassa (< 0.88) può riassumere fatti correlati ma indipendenti.

Spegni quando? & Soglie: Spegni quando i trace grezzi completi devono essere conservati ininterrotti per un audit forense. . Soglie: Conservazione: < 7 giorni = perdita di tracce importanti di audit . . > 60 giorni = grande DB SQLite (ore: 14 giorni). Soglia di Pruning: 0.95 = conservante . .88 = aggressivo (ore: 0.92).

85. Agente web visivo con audit screenshot (backend/ visual_web_agent .py)

Problema: Le attività su portali web complessi senza QX0ZQ (SPA, moduli, dashboard interattivi) non possono essere servite con gli scanner di testo.

Meccanica: Controlla un vero browser cromo headless (POST /api/browser/browse-and-capture), registra screenshot per ogni fase di clic e di input in .hermes/ visual_traces /{ task_id }/ ed estrae dati strutturati.

Benefici: Impronte di visibilità visiva e interazione automatizzata con applicazioni web moderne senza accesso API.

Prezzo & lato ombra (trade-off): Il cromo Headless richiede ~300 MB RAM e ~3 .05 secondi per ogni corsa di audit attiva.

Spegni quando? & Soglie: Spegnere quando? In caso di funzionamento puro backend senza controlli UI.

86. Test di stress della War Room virtuale e della Pre-Mortem (backend/ war_room_simulator .py)

Problema: I rischi di sicurezza, accettazione o conformità imprevisti sono spesso notati dopo il rollout durante l'introduzione dei prodotti.

Meccanica: Simula tre parti avversarie (critico, Pentester Observator, auditor di conformità rigoroso) prima della loro liberazione (POST /api/simulation/war-room). Crea una matrice di rischio aggregata (LOW a CRITICAL) con obblighi vincolanti.

Benefici: Individuare in modo proattivo le vulnerabilità e evitare costosi errori di decisione prima di andare in diretta.

Prezzo & lato ombra (trade-off): Costa ~1.500 ..3.000 token aggiuntivi e da +4 a +10 secondi. Tempo di attesa prima dell'attività vera e propria. La soglia MEDIUM è caratterizzata da un numero maggiore di falsi allarmi.

Spegni quando? & Soglie: Spegnere Auto Pre Mortem durante il normale giorno di sviluppo e utilizzarlo solo per gli interventi critici di produzione (HIGH / CRITICAL). . Soglie: soglia di blocco: MEDIUM = blocca spesso a titolo precauzionale . . CRITICAL = ferma solo i rischi catastrofici (h: HIGH).

87. SLA - contratti di consegna tra agenti (backend/ sla_contract_enforcer . py)

Problema: Le lotti di delegazioni in gerarchie multi-agente spesso portano a lavori incompleti che vengono scalati al QX0ZQ umano.

Meccanica: Conclude contratti digitali con campi obbligatori, lunghezza minima e criteri di accettazione per le delegazioni interne (POST /api/contracts/create). Valida le consegne in modo autonomo (POST /api/contracts/validate) e rifiuta automaticamente di correggere i risultati difettosi.

Benefici: Rispetto rigoroso degli standard di qualità nel team degli agenti ed evitare i livelli intermedi incompiuti nella scheda HITL.

Prezzo & lato ombra (trade-off): Se la lunghezza minima è troppo alta (> 300 caratteri), le risposte brevi precise vengono respinte e si innescano costosi cicli di lavoro di aggiornamento.

Spegni quando? & Soglie: Spegnere il reject automatico quando i subagente possono anche restituire risposte di stato o di sì/no brevi. . Soglie: min. Lunghezza: < 30 = lascia il segnaposto attraverso . > 300 = rifiuta risposte di fatto (ore: 100). Max Turnaround: ore 60 min.

88. Controllo centrale di autonomia e soglia (backend/ autonomy_settings .py)

Problema: Le soglie fisse nel codice del programma impediscono un rapido adeguamento delle regole di sicurezza, sandbox e resilienza in uso.

Meccanica: Raggruppa tutti i 10 gruppi di autonomia (Concurrency, Loop-killer, Autoguarigione, Docker-Sandbox, Time-Travel-Snapshot, Consenso-Gate, War Room, SLA-Contratti, Inbox/Standup e Igiene della flotta) in settings.json e sincronizzarli tramite GET / POST /api/settings/autonomy-modules con il QX0ZQ .

Benefici: Ogni modulo di protezione e autonomia può essere attivato o disattivato per la durata del modulo e può essere regolato con precisione al profilo di rischio e risorse desiderato.

Prezzo & lato ombra (trade-off): Un container difettoso può cadere in un loop crash a causa del ripristino automatico; l'audit visivo del browser lancia Chromium (+300 MB RAM , +4 secondi durata).

Spegni quando? & Soglie: Riavvio automatico durante la manutenzione manuale dei container; Disattivare l'audit del browser in caso di funzionamento di backend puro. . Soglie: giorni di backup DB: < 3 giorni = storia breve . . > 30 giorni = ~500 MB+ memoria di backup (ore: 7 giorni).

13. Performance-core, scalazione della portata e efficienza dei token

89. Caching semantico e rapido (backend/ semantic_cache .py)

Problema: Le richieste di agenti ricorrenti o leggermente riformulate causano inutili costi LLM - API e tempi di attesa lunghi seconde.

Meccanica: Calcola un hash SHA -256 per ogni prompt e un vettore L2 standardzzato per 3 grammi di parola in SQLite e fornisce immediatamente l'incontro di risposta salvato con un'equivalenza di Cosina ≥ 0.95.

Benefici: Tempo di risposta inferiore a 10 millisecondi e risparmio token 100 per le richieste semanticamente identiche.

Prezzo & lato ombra (trade-off): Se la soglia di Cosina è troppo bassa (< 0.92), la cache trascura le differenze di significato (ad esempio 'test-DB', "test cancella" vs 'prod-DB') e fornisce risposte false o obsolete.

Spegni quando? & Soglie: Disattivare le attività con dati in tempo reale (corsi di borsa, log live, stato del sistema) o quando ogni risposta deve essere generata di recente. . Soglie: soglia di Cosina: < 0.92 = risposte sbagliate dalla cache! . . > 0.98 = appena hit (ore 0.95). TTL: ore 24. Max Iscrizione: ore 500.

90. Map-Reduce parallela Sub-Tasking / Fan-Out & Fan-In (backend/ map_reduce_executor .py)

Problema: La gestione di molti documenti o di alcuni aspetti porta alla crescita lineare del tempo e a lunghi tempi di attesa.

Meccanica: Divide automaticamente gli ordini di lista fino a N, lavorando in parallelo ad Asyncio (semaforo-controllato) e raccoglie i risultati parziali nel passo Reduce.

Benefici: Moltiplica la portata delle analisi di documentazione, ricerca e copyright senza sovraccaricare il loop degli eventi.

Prezzo & lato ombra (trade-off): Genera N volte il carico di API (pericolo di HTTP -429 limiti di tasso nel provider) e ~20--35 % di overhead token attraverso la sintesi successiva di Reduce.

Spegni quando? & Soglie: Spegnimento con limiti di velocità rigorosi di API o quando i compiti parziali sono strettamente correlati e devono essere strutturati in sequenza. . Soglie: Max Worker: > 8 = Rischio di velocità-limite (ora: 4). Timeout di Chunk: ore 30s. A partire da elementi: ore 3.

91. Speculative fast-routing & latenza (backend/ speculative_hedger .py)

Problema: Le singole richieste di cloud (tail latency spike) bloccano l'intero processo di esecuzione degli agenti.

Meccanica: Dopo la scadenza di hedge_delay_seconds (ad esempio 2.5s), avvia automaticamente una chiamata parallela di sostituzione su un modello di fallback veloce e prende il risultato iniziale.

Benefici: Elimina i sospensione attraverso i punti terminali di API lenti e stabilizza i tempi di risposta al 99o decimale.

Prezzo & lato ombra (trade-off): Se il modello primario risponde subito dopo l'avvio, entrambe le richieste vengono fatturate (fino a 2x di costi token per i prompt lenti).

Spegni quando? & Soglie: Spegnimento con un budget API limitato o con modelli di resa lenta che richiedono tempo di riflessione > 10 secondi. . Soglie: ritardo hedge: < 1.5s = costi di calcolo quasi sempre doppi! . . > 8s = scarsa protezione latenza (ore 2.5s).

92. Compaction dinamica context & Summarizing gerarchico (backend/ context_compactor .py)

Problema: Le lunghe sessioni di agenti gonfiano la finestra di contesto, rallentano il prompt prefill e aumentano drasticamente i costi per turno.

Meccanica: Estrae tutti i fatti difficili (obiettivi, percorsi, decisioni, errori) da un blocco compatto [HIERARCHICAL_MEMORY_STATE] e mantiene solo i giri più recenti nel testo completo.

Benefici: Salva 65 .80 % di token di contesto per le sessioni lunghe senza perdere i dati di progetto importanti.

Prezzo & lato ombra (trade-off): La corsa di compressione costa ~1.000 token una volta; le righe di codice esatte o i piccoli dettagli di messaggi passati possono essere persi nel sommario.

Spegni quando? & Soglie: Spegnere se l'agente deve letteralmente ricorrere a messaggi molto in anticipo della stessa sessione. . Soglie: soglia di token: < 3000 = dimentica dettagli troppo presto . . > 24000 = costo elevato di rapida esecuzione (ora: 6000).

93. SQLite WAL Connection Pool & Async Write-Batching (backend/ async_write_batcher .py)

Problema: Centinaia di singoli INSERT SQLite in esecuzione di operazioni di agenti paralleli generano punte di disco I/O e 'database is locked' conflitti.

Meccanica: Raccoglie gli eventi di telemetria e audit in un anello RAM sicuro e thread e li scrive in bundle via executemany() in modalità WAL.

Benefici: Riduce l'accesso al disco rigido di circa l'80% e mantiene il loop principale dell'evento senza blocchi.

Prezzo & lato ombra (trade-off): In caso di blackout duro o di kill -9 prima dell'intervallo di rotazione successivo, le voci tampone RAM perdono gli ultimi 1 .05 secondi.

Spegni quando? & Soglie: Spegni se ogni singola voce di log deve essere sincronizzata sul disco rigido immediatamente e senza ritardo di secondi. . Soglie: Dimensione batch: < 5 = molti writes di disco . . > 100 = più perdita in caso di crash (ore 25). Intervallo di colore: h 2.0s.

14. Architettura moderna di memoria & RAG

94. Search ibrido con BM25 FTS5 & Reciprocal Rank Fusion (backend/ hybrid_search_engine .py)

Problema: La ricerca di vettori puri trova testi semanticamente simili, ma spesso non riesce a trovare codici di errore precisi, nomi di funzione o numeri di serie.

Meccanica: Combina SQLite FTS5 ranking a testo intero (BM25) e somiglianza del vettore cosine tramite Reciprocal Rank Fusion (RRF).

Benefici: Unisce l'esatto appuntamento di parole chiave e la comprensione semantica in un unico elenco di hit di

alta precisione.

Prezzo & lato ombra (trade-off): La ricerca di un indice doppio aumenta la latenza di ricerca di ~15 .40 ms; un top-K troppo alto (> 10) riempie il prompt dell'agente con migliaia di token aggiuntivi.

Spegni quando? & Soglie: Quasi mai spegnere .Mai al massimo top K a 3 .05 se si vuole ridurre al minimo i costi dei token per richiesta. . Soglie: costante RRF K: h 60. BM25 Peso: 0.0 = solo vettore .0.5 = equilibrio .0 = solo parole chiave (h: 0.50).

95. Archivio locale di Knowledge / Tripel (backend/ knowledge_graph_store .py)

Problema: I hunk di testo piane non rilevano le dipendenze strutturate (chi gestisce quale sistema o componente utilizza quale servizio).

Meccanica: Estrae e salva Tripoli di oggetto predicato soggetto con punteggio di confidenza in SQLite e permette traversate grafiche multi-livello (multi-Hop).

Benefici: Gli agenti riconoscono i legami tra sistema e organizzazione su più livelli.

Prezzo & lato ombra (trade-off): L'estrazione automatica di triplo costa ~500 token aggiuntivi. Da > 2 Traversal-Hops esplode esponenzialemente il numero di nodi vicini.

Spegni quando? & Soglie: Spegnere l'estrazione automatica con chat a breve termine pura senza acquisizione di conoscenze a lungo termine; limitare i Hop a un massimo di 2. . Soglie: Max Hops: 1 = vicini diretti . . 2 = ottimale . . ≥ 3 = esplosione di nodi! (h: 2). Min. Confidenza: ore 0.70.

96. Agentic Corrective QX0ZQ / CRAG (backend/ corrective_rag .py)

Problema: QX0ZQ classico passa al modello vocale non testato per gli obiettivi di ricerca non pertinenti, provocando così risposte errate.

Meccanica: Valuta ogni chank del documento recuperato contro un limite di rilevanza (0.45) e, in caso di hit debole, esegue automaticamente un rewrite query con espansione sinomonica.

Benefici: Filtra il rumore della conoscenza prima della generazione di risposte e aumenta la qualità del hit in caso di domande poco chiare.

Prezzo & lato ombra (trade-off): Ogni riformulazione Query costa 1 chiamata LLM aggiuntiva (+1,5 - +4 secondi. Latenza). Se la soglia di rilevanza è troppo alta (> 0.70), CRAG ruota giri supplementari inutili.

Spegni quando? & Soglie: Spegnere quando il database delle conoscenze è piccolo e pulito o richiede tempi di risposta rapidi sotto i 2 secondi. . Soglie: soglia di rilevanza: < 0.30 = lascia sorgenti deboli attraverso . . > 0.70 = cicli di ricerca infiniti (h: 0.45). Max Rewrites: h 2.

97. Memoria di Core a 2 livelli secondo l'immagine MemGPT/Letta (backend/ core_memory_manager .py)

Problema: Le principali preferenze degli utenti e gli obiettivi del progetto vengono persi non appena vengono eliminati dalla finestra di chat di breve durata.

Meccanica: Blocchi di memoria di lavoro di Pinned-Working modificabili (< CORE_WORKING_MEMORY >) permanenti nel prompt del sistema, separati dalla memoria di archivio di grandi dimensioni.

Benefici: Garantisce una memoria permanente, che può essere raccolta dall'agente stesso, per un numero di sessioni illimitato.

Prezzo & lato ombra (trade-off): Ogni blocco di spinto occupa costantemente il carico base di 2.000 token per messaggio in ogni chiamata LLM (10 blocchi × 800 caratteri).

Spegni quando? & Soglie: Spegnere o ridurre il numero di blocchi a 3 .05 quando si utilizzano piccoli modelli con finestre di contesto strette o brevi bozzetti di gettoni. . Soglie: Max Blocchi: > 15 = consumo di gettoni di base elevato (ore: 10). Max Segno/blocco: < 300 = taglia le regole a partire da . . > 1500 = Prompt ostruito (ora: 800).

98. Pinning automatico di citazione e sorgente (backend/ citation_pinner .py)

Problema: Senza riferimenti diretti a fonti, l'utente non può verificare da quale documento proviene una dichiarazione di fatto dell'agente.

Meccanica: Aggiunge le frasi della risposta tramite token-overlap ai documenti di origine recuperati, aggiunge la composizione in linea ([Fonte: ID]) e controlla la percentuale minima di occupazione.

Benefici: Rendete ogni dichiarazione comprensibile e immediatamente rivelate le affermazioni infondate.

Prezzo & lato ombra (trade-off): Aumenta il consumo di token di output del ~10 ..15 %; una copertura minima troppo elevata (> 80%) provoca un errore di allarme in caso di problemi creativi o di sintesi.

Spegni quando? & Soglie: Spegnimento in caso di brainstorming libero, generazione di codici o chat di routine senza riferimento ai documenti. . Soglie: Min. copertura: < 30% = dichiarazioni in bianco . . > 85% = allarmi errati in caso di sintesi (h: 60%).

15. Esecuzione e determinazione affidabile (Durable Execution)

99. Garanzia schema Structured output & JSON (backend/ structured_output_enforcer .py)

Problema: I modelli linguistici talvolta forniscono QX0ZQ non validi (frame a discesa, parentesi mancanti o tipi di dati errati), con conseguente caduta di pipeline di codice a valle.

Meccanica: Ripara automaticamente gli errori di sintassi nella stringa grezza e impone tutti i campi obbligatori e i tipi di dati dello schema di destinazione passato.

Benefici: Garantito 100% di payload JSON a prova di tipo tra agenti e interfacce backend.

Prezzo & lato ombra (trade-off): Ogni ciclo di riparazione auto costa da +1 a +3 secondi. Latenza e token aggiuntivi. Convalida schema strikte rifiuta le risposte anche a causa di campi aggiuntivi innocui.

Spegni quando? & Soglie: Disattivare la convalida stritta quando gli endpoint o gli script API possono anche gestire campi JSON flessibili o parziali. . Soglie: Prova schematica: Strikt vs. Tolerant (h: Strikt). Max Riparazioni: 1 .62 = riparato 98 % di tutti gli errori di battitura . . > 3 = sprecato token (h: 2).

100. Durable Execution Checkpointing / Pausa & Resume (backend/ durable_checkpoint_manager .py)

Problema: Se un ordine di ricerca o di build a 10 fasi si conclude con un nuovo avvio al punto 8, tutti i 7 lavori preparatori sono stati finora ripetuti a costi elevati.

Meccanica: Dopo ogni fase completata, assicura la completa variabile e stato di avanzamento transazionale in SQLite.

Benefici: I corridori interrotti rimontano esattamente all'ultimo checkpoint valido senza dover consumare il doppio del lavoro o dei token.

Prezzo & lato ombra (trade-off): Il checkpointing in ogni singola fase (N = 1) genera un notevole blast di scrittura SQLite. Il recupero automatico durante l'avvio può riaccendere involontariamente le applicazioni obsolete dal giorno precedente.

Spegni quando? & Soglie: Spegnere il recupero automatico se le attività interrotte dopo un riavvio devono essere verificate e rilasciate manualmente. . Soglie: Tutti i passaggi N: 1 = assolutamente sicuro da crash, più disco I/O . . 3 . 5 = risparmia SSD (ora: 1). Conservazione: ore 14.

101. Deduplicazioni di chiamata di idepotency & strumento (backend/ idempotency_guard .py)

Problema: In caso di rese di rete o di cicli di agente, vi è il rischio che la stessa e-mail, fattura o azione Webhook venga attivata più volte.

Meccanica: Per ogni chiamata di strumento che cambia, assegna una chiave SHA -256-Idemopotency al lettore SQLite e blocca le chiamate di follow-up identiche all'interno di TTL.

Benefici: Previene in modo affidabile la doppia incidenza esterna, seguendo il modello dei moderni gateway di pagamento (principio Stripe).

Prezzo & lato ombra (trade-off): Se l'utente desidera eseguire intenzionalmente lo stesso comando entro il periodo di blocco (standard 24 h), la guardia blocca.

Spegni quando? & Soglie: Riduci le ore TTL a 1 ora o spegnile per un breve periodo se le stesse azioni devono essere eseguite più volte consecutive durante il test. . Soglie: TTL (ore): < 1h = protezione breve dei duplicati . . > 48h = ripetizione bloccata voluta (ore 24).

102. Dry-Run / Shadow Mode per gli ordini degli agenti (backend/ dry_run_simulator .py)

Problema: Gli ordini di agenti nuovi o complessi spesso non si vogliono provare prima di eseguire con precisione i

dati di produzione reali senza rischi.

Meccanica: Nella modalità Shadow, esegue tutti i comandi di file, database e API scriventi e registra invece un piano di anteprima dettagliato delle mutazioni.

Benefici: Consente di eseguire esercizi di essiccazione e audit di rischio senza rischi prima della loro messa a disposizione definitiva.

Prezzo & lato ombra (trade-off): SOLANGE AKTIV , WERDEN KEINE ECHTEN DATEIEN GESCHRIEBEN UND KEINE ECHTEN APIS! Se si dimentica l'interruttore, il sistema agisce verso l'esterno 'tot'.

Spegni quando? & Soglie: Lasciare obbligatoriamente AUSGESCHALTET in normale funzionamento produttivo! Attiva solo per esercizi di essiccazione (dri-runs) e audit di sicurezza. . Soglie: interruttore principale per impostazione predefinita AUS (false). Abilita solo per i test isolati.

103. Letter dead (DLQ) con gestore di riproducitore automatico (backend/ dead_letter_queue .py)

Problema: I Webhook non riusciti o i lavori di background asincroni vengono persi senza lasciare traccia in caso di incidenti di breve durata di terze parti senza lavagne.

Meccanica: Salva ogni lavoro fallito, con payload originale e motivo di errore nella tabella DLQ e lo ripete con backoff esponenziale.

Benefici: Nessun ordine viene perso; le task non riuscite possono essere riprodotta in qualsiasi momento singolarmente o raccolti.

Prezzo & lato ombra (trade-off): In caso di ordini non corretti (ad esempio, chiave API non valida), le prove di ripetizione consumano risorse inutili; i ripetitori con ritardo possono attivare azioni in ritardo.

Spegni quando? & Soglie: Spegnere il replay automatico quando i lavori falliti devono essere visti solo manualmente e rilasciati singolarmente. . Soglie: Max Retries: 1 = si rompe in caso di guasto recente . . > 5 = stacca i lavori morti (ora: 3). Base di backoff: ore 10 (10s -> 20s -> 40s).

16. Sicurezza, protezione dei dati e guardrails dell'Enterprise

104. Scrubber & Re-idratazione bidirezionale PII (backend/ pii_scrubber .py)

Problema: Quando e-mail, numeri di telefono o IBAN vengono trasmessi in chiaro a LLM cloud esterni, GDPR minaccia di non essere utilizzato e di flussi di dati indesiderati.

Meccanica: Sostituisce i dati personali sensibili prima della chiamata API localmente da segnaposto ([EMAIL_1], [IBAN_1]) e li scambia nuovamente con i valori reali nella risposta ricevuta.

Benefici: I modelli cloud non vedono mai dati personali veri, mentre gli utenti sul posto ricevono documenti e risposte complete.

Prezzo & lato ombra (trade-off): Se il QX0ZQ cambia leggermente il placet nel testo della risposta, la traduzione di ritorno (riidratazione) fallisce. Inoltre, la mascheratura Regex può a volte alterare le stringhe di codice.

Spegni quando? & Soglie: Spegni quando vengono utilizzati solo modelli locali (Ollama) o quando il QX0ZQ deve necessariamente analizzare i veri domini/ nomi email. . Soglie: Interruttore granulometrico per e-mail, numeri di telefono, IBAN e reidratazione automatica (tutti attivi per impostazione predefinita).

105. Egress-Firewall & Domain-Allowlisting per agenti (backend/ egress_firewall .py)

Problema: Attraverso le injections di prompt nascoste nei siti web, gli agenti potrebbero essere incaricati di inviare i dati interni ai server di aggressori stranieri tramite HTTP.

Meccanica: Controlla ogni destinazione in uscita URL contro l'Allowlist di dominio configurato e blocca gli host non autorizzati e gli indirizzi IPv4 grezzi.

Benefici: Interrompe l' estrapolazione dei dati a livello di rete anche quando si manipolano i dati.

Prezzo & lato ombra (trade-off): Finché allowlist_only è attivo, qualsiasi nuova pagina web, QX0ZQ o documentazione che non sia esplicitamente inclusa nella lista di dominio viene bloccata duramente nelle ricerche web.

Spegni quando? & Soglie: Spegni (o completi i domini necessari) se l'agente di ricerca deve effettuare ricerche libere sul web aperto. . Soglie: Domini autorizzati: Whitelist separato da virgola. Blocco IP grezzo protegge dalle escrezioni

IP dirette.

106. Agents granulonari- RBAC & File-Scope-Jails (backend/ agent_rbac_jail .py)

Problema: Se ogni sub-agente ha pieno accesso alla cartella del progetto, un agente di frontend può accidentalmente sovrascrivere i file del kernel backend o la chiave .env.

Meccanica: Lega ogni ruolo di agente ad una lista fissa di strumenti e ad una jail di directory autorizzata (ad esempio static/ per gli agenti frontend).

Benefici: Applica il principio dei privilegi minimi (least price) per ogni singolo processo nel sistema operativo.

Prezzo & lato ombra (trade-off): Impedisce ad un amministratore di attendere i file di configurazione legittimi al di fuori della cartella del progetto o di controllare direttamente i modelli .env.

Spegni quando? & Soglie: Solo per le attività di manutenzione del sistema dedicate, sciogliere per breve tempo; la protezione .env deve rimanere attiva in modo normale. . Soglie: Interruttori per Path-Jail, .env-Sperre e Sperre (/etc, /root, /proc).

107. Checker & policy output deterministico (backend/ output_policy_linter .py)

Problema: I modelli vocali si dissolvono occasionalmente nonostante il prompt del sistema in flacci artificiali di IA o generare snippet di shell rischiosi.

Meccanica: Scansiona il testo generato in tempo reale usando espressioni regolari, rimuove l'open/album di Sycophancy e disinnesca i comandi distruttivi.

Benefici: Garantisce precisione linguistica e sicurezza tecnica senza ulteriore chiamata di prova LLM.

Prezzo & lato ombra (trade-off): Il filtro distruttivo può anche bloccare innocui script di pulizia o le migrazioni di database che si intendono effettuare quando vi sono comandi come DROP o rm -rf.

Spegni quando? & Soglie: Disabilita temporaneamente i bloccanti distruttivi quando vengono creati script di pulizia del server o le migrazioni di reset SQL. . Soglie: Rimuovi interruttori per le discoteche AI, blocchi i comandi distruttivi e pulizia automatica.

108. Receipt di sicurezza di manipolazione / SHA -256-Qritture (backend/ execution_receipt_ledger .py)

Problema: Per gli audit di conformità (EU AI Act / ISO 42001) i log semplici di testo non sono sufficienti, poiché possono essere modificati successivamente.

Meccanica: Concatena ogni calcolo delle task (modello, token, costo USD, strumenti eseguiti) con l'hash SHA -256 della voce precedente in una catena simile a Blockchain.

Benefici: Qualsiasi successiva manipolazione dei protocolli di costo o di decisione è immediatamente nota durante la verifica dell'integrità.

Prezzo & lato ombra (trade-off): Provoca un'overhead minima di calcolo e archiviazione (~2 ms e ~2 KB SQLite per ogni task); la catena di hash cresce continuamente per mesi.

Spegni quando? & Soglie: Può rimanere attivo in modo permanente durante il funzionamento produttivo; disattivare solo su microsistemi estremamente limitati di memoria. . Soglie: Incatenamento Hash (QX0ZQ) + ripartizione dettagliata dei costi per corsa.

17. Auto-ottimizzazione, valutazione e intelligenza dello sciame

109. Ottimizzatore automatico di rapida esecuzione secondo il principio DSPy/TextGrad (backend/ prompt_optimizer .py)

Problema: I prompt statici del sistema continuano a ripetere gli stessi errori finché nessun uomo corregge manualmente.

Meccanica: Se valorizza i log di errore reali di un ruolo, individua le debolezze sistematiche e genera regole di aggiunta mirate per il prompt del sistema.

Benefici: Gli agenti imparano continuamente da gare fallite e induriscono le loro istruzioni in modo autonomo.

Prezzo & lato ombra (trade-off): Se Auto-Apply è attivo, una regola derivata difettosa può distorcere in modo

permanente il comportamento dell'agente (Drift Prompt) e gonfiare il prompt del sistema.

Spegni quando? & Soglie: Lasciare spegnere l'apply in modalità produttiva per far sì che le patch prompt proposte siano accettate solo dopo la prova umana. . Soglie: A partire da errori: 1 = esagerato a errori singoli . . > 8 = imparerà molto lento (h: 3). Max Regole: > 10 = gonfia il prompt del sistema (h: 5).

110. Suite di prova di regressione standard Gold / Eval Runner (backend/ eval_regression_runner .py)

Problema: Un presunto piccolo cambiamento nel prompt o modello del sistema può peggiorare in modo inosservato le regole di sicurezza o di formato esistenti.

Meccanica: Prima di assumere nuove assunzioni, esegue test automatizzati sull'oro standard e verifica il tasso di esistenza e i campioni vietati.

Benefici: Previene le regolazioni di qualità in funzionamento produttivo da un gate di qualità obiettivo.

Prezzo & lato ombra (trade-off): Ogni flusso completo di Eval lancia tutti i prompt di test depositati contro il LLM - API (a seconda della portata del test ~5.000 .25.000 token e 15 .60 secondi).

Spegni quando? & Soglie: Spegnere il blocco di defloy quando i test sperimentali rapidi sono effettuati senza tempo. . Soglie: Min. Pass rate: < 75% = lascia il deterioramento attraverso . . 100% = bloccato già a variazioni minori (ora: 90%).

111. router multi-armato di bandito modello (backend/ bandit_model_router .py)

Problema: Le istruzioni fisse per il modello non rispondono quando un fornitore di cloud si rallenta improvvisamente o mostra un aumento dei tassi di errore.

Meccanica: Punteggia ogni modello in base al tasso di successo reale (60%) e alla velocità (40%) e sceglie dinamicamente il miglior candidato.

Benefici: Massimizza automaticamente la qualità della risposta e il ritmo grazie alla telemetria vera e propria.

Prezzo & lato ombra (trade-off): Il router, con il tasso di esplorazione (psilon, ad esempio 0.15 = 15%), testa intenzionalmente modelli alternativi che possono essere più lenti in un caso specifico.

Spegni quando? & Soglie: Spegnere (o impostare Epsilon a 0.00) se si vuole imporre un modello fisso e prevedibile per ogni corsa, 100 per cento. . Soglie: Epsilon: 0.00 = usa ostinatamente il modello migliore . . > 0.30 = prova troppo spesso modelli più deboli (ore 0.15). Peso successo/latenza: ore 0.60 / 0.40.

112. Spawning dinamico del team efemero (backend/ ephemeral_team_spawner .py)

Problema: Un singolo agente generale è rapidamente sopraffatto da grandi funzioni trasversali (architettura + QX0ZQ + sicurezza + QA).

Meccanica: Recluta una task force temporanea di specialisti e la reintegra dopo la missione compiuta.

Benefici: Raccoglie le conoscenze specifiche in base alle esigenze senza una copertura permanente delle risorse.

Prezzo & lato ombra (trade-off): Un team ad hoc di 4 persone moltiplica il consumo di token (3x-5x) e richiede un'ulteriore coorte di coordinamento tra i membri del team.

Spegni quando? & Soglie: Spegnere i compiti singoli o il budget di token ridotto; limitare le dimensioni massime del team a 3 . . Soglie: Max Team taglia: 2 .4 = ideale . . > 5 = alto consumo di token e comunicazioni (h: 4).

113. Protector & host delle risorse dal vivo (backend/ resource_backpressure .py)

Problema: I container paralleli, gli audit del browser e i processi locali possono sovraccaricare RAM o CPU (kill OOM) in caso di picchi di carico.

Meccanica: Sorveglia in tempo reale le metriche CPU, RAM e disco e riduce automaticamente i task di sfondo a basso livello di priorità a partire dall'85% di utilizzo.

Benefici: Protegge il server in modo affidabile da cadute dovute a sovraccarico e mantiene in ogni momento i processi essenziali critici reattivi.

Prezzo & lato ombra (trade-off): Se le soglie sono troppo basse (CPU < 70 % o RAM < 70 %), le operazioni di background vengono frenate costantemente anche in normali picchi di carico brevi.

Spegni quando? & Soglie: Non completamente disattivare . . Invece, con hardware forte, aumentare le soglie CPU -/ RAM al 90%. . Soglie: Max CPU / RAM: < 70% = riduce il rischio OOM al normale lavoro . > 92% = ora: 85%. min. disco

libero: ore 2.0 GB.

18° pilastro 3: Economia & QX0ZQ (alfa) & suite cliente

Modulo / View	Estensione delle funzioni e caratteristiche tecniche (QX0ZQ)
Indirizzi & QX1ZQ + QX0ZQ - Sync	Contatto centrale, debitori (10000+) e creditore (70000+), dracker a 360°, multi-canale, calendario/riproiettori e sincronizzazione LinkedIn & Media tramite QX0ZQ (Porte 305/8090/8092).
Articolo & Prestazioni (ART)	Catalogo dei dati a partire da ART -1001 per servizi, tariffe orarie, pacchetti, abbonamenti SaaS e prodotti fisici con gestione delle scorte.
Operazioni e contratti	Offerta catena di copertura continua (ANG - YYYY - NNNN) → Ordine (AB- YYYY - NNNN) compreso il gasdotto a deal ponderato, il forecast di vendite e il radar di consegna contrattuale (allerta 30/14/7 giorni).
Contabilità & QX0ZQ	Note di uscita e di entrata secondo l'articolo 14 UstG, QR di pagamento GiroCode, analytics finanziari, ingiunzione di pagamento automatica (Dunning Engine) e esportazione EXTF / CSV per QX0ZQ .
Isolamento corporate e cliente	Diverse imprese (company_id) con database SQLite separati fisicamente in modalità WAL, numeri, fogli di corrispondenza ed editor di modelli.

19. Caratteristiche architettoniche, di sicurezza e di deployment (QX0ZQ . QX1ZQ .)

Caratteristiche / Sottosistema	implementazione e benefici operativi (QX0ZQ)
Architettura modulare del router	Esternalizzazione in router FastAPI dedicati sotto arkhon/routers/ (main.py da 16.254 a 1.875 righe, -88 %) e strato di configurazione centrale arkhon/core/config.py.
Sicurezza Hardening (Archon Hardened)	Chiusura di 25 risultati critici e 68 alti audit; Scarico di threadpool Starlette, percorso UI-DOM indurito e proxy di Admin Docker di classe A per hermes_updater .
Gestione utente RBAC (17.10)	Ruote granulometriche (QX1ZQ , QX2ZQ , QX3ZQ) tramite backend/ auth_service .py con 100.000 giri QX0ZQ Hashing password e modalità singolo utente/multiutente.
Aggiornamento del sistema in 2 fasi (17.11)	Visualizzazione istantanea della versione locale/ HEAD via ? local_only =true (~0,25 s) più asincronizzato GitHub, pre-aggiornamento SQLite snapshot online e un click-rollback.
QX0ZQ Proxy & Chroma Stack	Incorporazione senza cuciture di QX0ZQ nella libreria centrale tramite /anithingllm-proxy incl. /assets/* Fast-Path e indice vettoriale locale QX1ZQ.
Installatore portatile da 1 clic	Installazione iniziale inter-piattaforma tramite setup.bat, setup.ps1 e install.sh con docker-compose.yml, .env.example, INSTALL_AGENT .md e 9-lingue GitHub-Dou.
Sandbox di qualità e codice a 4 occhi	Controllo automatico di seconda mano nel Kanban prima della conclusione del biglietto e sandbox interattivo nel QX0ZQ (4.4.1) per l'anteprima dal vivo.

20. Glossario alfabetico e indice analitico

Elenco alfabetico dei termini di tutti i moduli, componenti architettonici e caratteristiche (QX0ZQ) con numeri di pagina esatti.

Caratteristica / termine	Breve descrizione	Pagina (n)
3 animali costo-routing	Selezione automatica del modello in LOW / MEDIUM / HIGH per complessità delle attività e budget.	S. 3, 8
4 occhi di qualità gate	Esame indipendente delle mansioni di Kanban completate da un agente di Revisor.	S. 3, 14, 31
Anti-AI-Slop & STE100	Filtri di stile e parole per evitare inutili flosce e risparmiare token di uscita.	S. 8, 10
Approval Gate (QX0ZQ)	Passo di verifica umano prima di e-mail esterne, prenotazioni o accesso a file critici.	S. 4, 17, 22, 28
Architettura del router (arkhon/routers)	Ripartizione modulare del backend in router FastAPI sottile (-88 % di dimensione del file in main.py).	S. 3, 31
Arcon Hardened (QX0ZQ)	La versione di sicurezza e stabilità con struttura modulare del router e risultati di audit chiusi.	S. 3, 31
Bot screen (navigatore web)	Monitor di esecuzione video per le interazioni del browser via Playwright.	S. 14, 23
Capacità di corporate & cliente	Separazione fisica di più società tramite company_id .	S. 3, 31
Catena di composizione ERP (ANG -> AB -> RE)	Documenti di vendita e contabilità collegati, compresi i servizi di trasporto di cavi e di ingiunzione di pagamento.	S. 22, 24, 25, 31
Compaction & LRU - K di Context	Spostamento di memoria e compressione di lunghi contesti di chat al raggiungimento del limite RAM.	S. 8, 9, 24, 25
Consolidamento Sleep-Cycle	Deduplicazioni notturne e compattazione a lungo termine della memoria degli agenti.	S. 15, 23
Contratti SLA (Contract Enforcement)	Consegna formale e garanzia di qualità tra agenti specializzati.	S. 13, 23, 24
Corrective QX0ZQ (CRAG)	Domanda di conoscenza autocontrollata con valutazione della pertinenza prima della generazione di risposte.	S. 25, 26
DATEV -Export & GiroCode	EXTF / CSV - Interfaccia per i dati contabili e QR di pagamento EPC sulle fatture.	S. 3, 31
DSPy / TextGrad Optimizer	Migliorare automaticamente i prompt dei sistemi utilizzando metriche di valutazione reali.	S. 29
Firewall di Egress	Controllo di rete e dominio in uscita per chiamate in too-up da agenti.	S. 28
Grafico della conoscenza	Memoria locale triple per relazioni strutturate di entità oltre ai dati vettoriali.	S. 25, 26
Hermes Master Orchestrator	Centrale centrale per la separazione, la delegazione e la sintesi del demone su porta 8420.	S. 9, 11, 12, 19
JEV Router (0 token Intent)	Prefiltri deterministici senza costi IA per scorciatoie uniche.	S. 8, 10
Keys & Dry-Run	Protezione contro la doppia esecuzione degli strumenti e modalità di simulazione senza accesso alla scrittura.	S. 27, 28

Caratteristica / termine	Breve descrizione	Pagina (n)
killer QX0ZQ & Loop	Protezione in tempo reale contro i cicli senza fine (prova Jaccard) e i superamenti dei token.	S. 3, 6, 8, 11
Log di audit (Hashcane QX0ZQ)	Catena di protocollo sicura per tutte le azioni di sistema e di agente.	S. 13, 17, 28, 29
PBKDF2 / QX0ZQ QX1ZQ	Backup di rotolo e password nel QX0ZQ (tab. 17.10).	S. 3, 28, 29, 31
Pre-mortem & Consenso Gate	Prova di stress simulata di errore e voto multi-agente prima delle autorizzazioni critiche.	S. 5, 21, 23, 24
Quea dead-letter (DLQ)	Serpente di attesa di errore con controllo automatico di ripetizione per lavori falliti.	S. 27, 28
QX0ZQ	Base di dati vettoriale locale per il recupero di documenti semantici nel secondo pilastro	S. 25, 31
QX0ZQ	Ricerca in più fasi su web e fonti con triangolazione di evidenza.	S. 3, 7, 21
QX0ZQ (11 schede)	Console centrale di amministrazione, con l'aggiunta di utenti/ QX0ZQ e aggiornamenti di sistema a 2 fasi.	S. 3, 8, 11, 31
QX0ZQ (55 frameworks)	Modulo strategico per dibattiti strutturati su boardroom su 9 categorie metodologiche.	S. 3, 5, 20
QX0ZQ (Model Context Protocol)	Architettura standard dei driver per la connessione sicura degli strumenti.	S. 6, 14
QX0ZQ / Hermes Plugins	Estensioni modulari di sistema per memoria, delegazione, verifica e interfacce.	S. 3, 19, 20, 21
QX0ZQ AI Gateway	Gateway multi-fornitore, incluso Free-Tier Routing, Latency-Hedging e Eval-Arena.	S. 3, 6, 21
QX0ZQ proxy	Incorporazione locale di QX0ZQ + QX1ZQ tramite /anythingllm-proxy nella libreria centrale.	S. 3, 31
router di modello Bandit	router adattivo (multi-armato) per la selezione dinamica del modello in base al tasso di hit e ai costi.	S. 29, 30
Sandbox Ephemeral Code	Tempo di fuga isolato per il codice di pitone e script sicuri.	S. 3, 23, 31
Scheduler DAG	Pianificazione di esecuzione in due fasi di compiti parziali dipendenti con fan-out / fan-in parallelo.	S. 12, 13, 24
Scrubber e re-idratazione PII	Anonimizzazione bidirezionale dei dati personali sensibili prima delle chiamate esterne LLM.	S. 3, 28
Search ibrido (FTS5 + RRF)	Combinazione di BM25 - Ricerca a testo intero e ricerca vettoriale tramite Reciprocal Rank Fusion.	S. 25
Semantic Caching	Cache 24h di contenuto simili prompt per il risparmio di token.	S. 8, 9, 24, 26
SQLite WAL piscina di collegamento	Accesso multi-thread al database sicuro da scrittura tramite Write-Ahead Logging.	S. 15, 20, 21, 24
Supervisor di Watchdog & Deadlock	Monitoraggio 30 secondi di task appesi, container e autoguarigione automatica.	S. 6, 11, 12, 21
Travel & time execution durable	I cicli di fondo interrompibili con checkpoint SQLite e replay di esecuzione graduale.	S. 3, 21, 27, 28
Trigger per eventi e webhook	Eventi di elaborazione di webhook e modifiche delle cartelle.	S. 14, 21, 22

Caratteristica / termine	Breve descrizione	Pagina (n)
Valutatori dei costi pre-flight	Calcolo anticipato dei costi di token previsti, compreso il costo del task-budget in caso di superamento.	S. 8, 10, 11